UPF

Distributed Search Engines

sharding, replication and query routing

Marc, Èric, Oriol

21 October 2026

What we will cover

Marc M

Problem Principles Example Trends More

The problem: one machine is not enough

Why an inverted index on one machine breaks at web scale, and why we add machines instead of buying a bigger one.

Main principles

Split the index by document, replicate the shards, scatter the query and merge the answers, without being fooled by local statistics or slow shards.

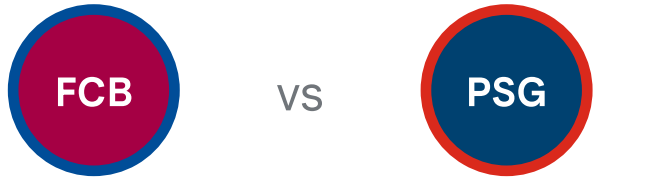
One query, step by step

A deliberately tiny cluster answers “lamine yamal”: cache, routing, local search, a slow machine, fresh pages, merge.

Trends and limits

What is matched is changing, who is searching is changing, and what stays the same.

Yesterday



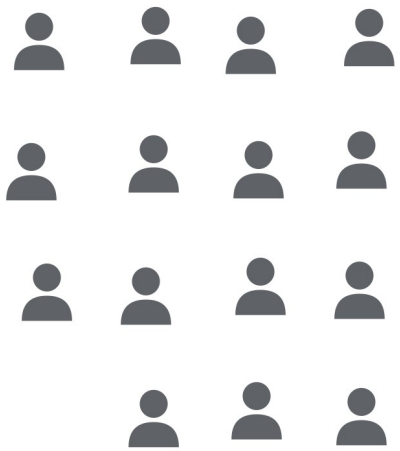
FC Barcelona Paris Saint-Germain

lamine yamal

lamine yamal

Marc M

Problem Principles Example Trends More



Popular

everyone types the same query

lamine yamal

Marc M

Problem Principles Example Trends More



Popular

everyone types the same query

All at once

thousands at the same moment

lamine yamal

Marc M

Problem | Principles | Example | Trends | More



Popular

everyone types the same query

All at once

thousands at the same moment

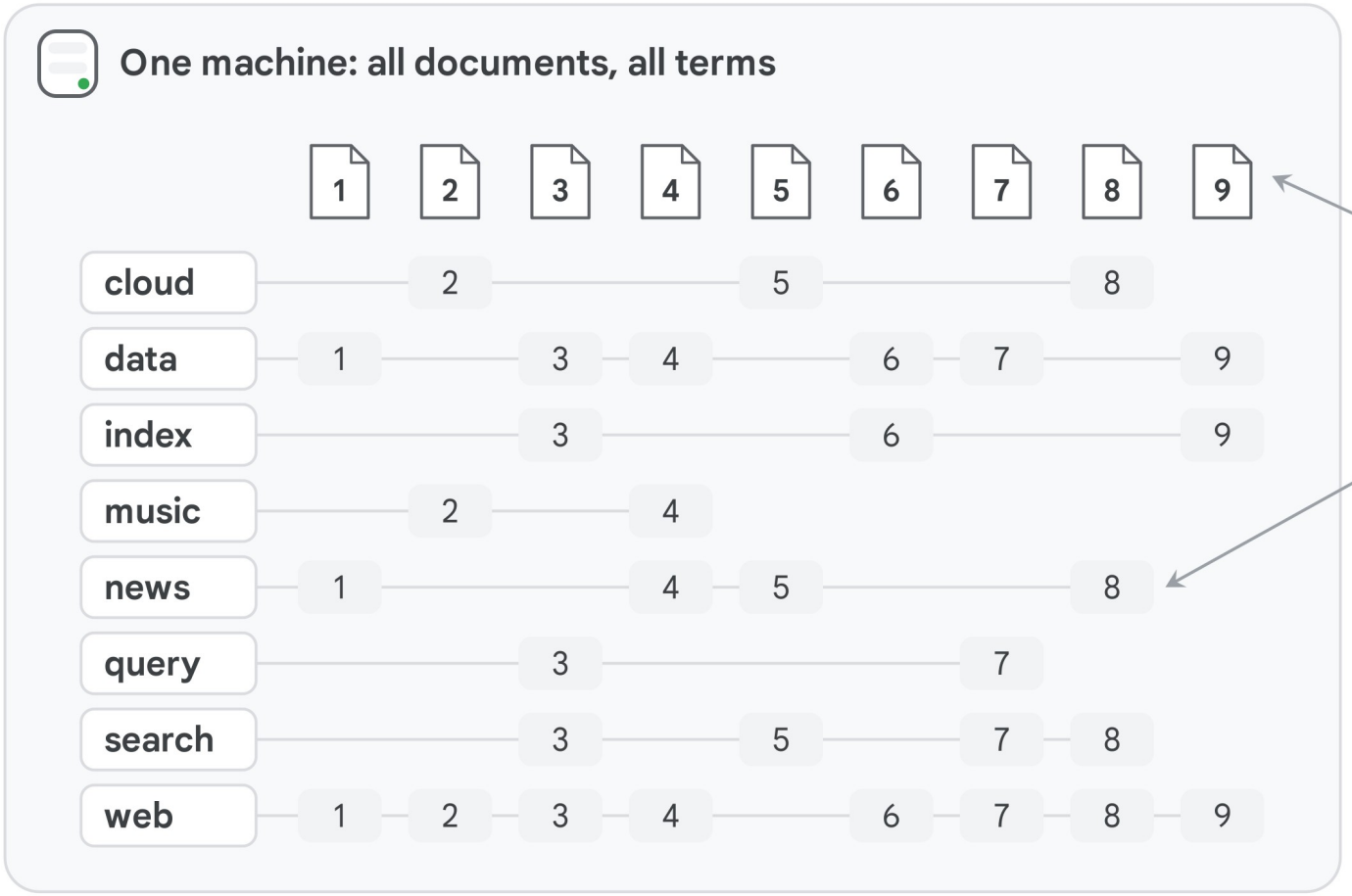
Fresh

pages written hours ago

One machine: the inverted index

Marc M

Problem | Principles | Example | Trends | More

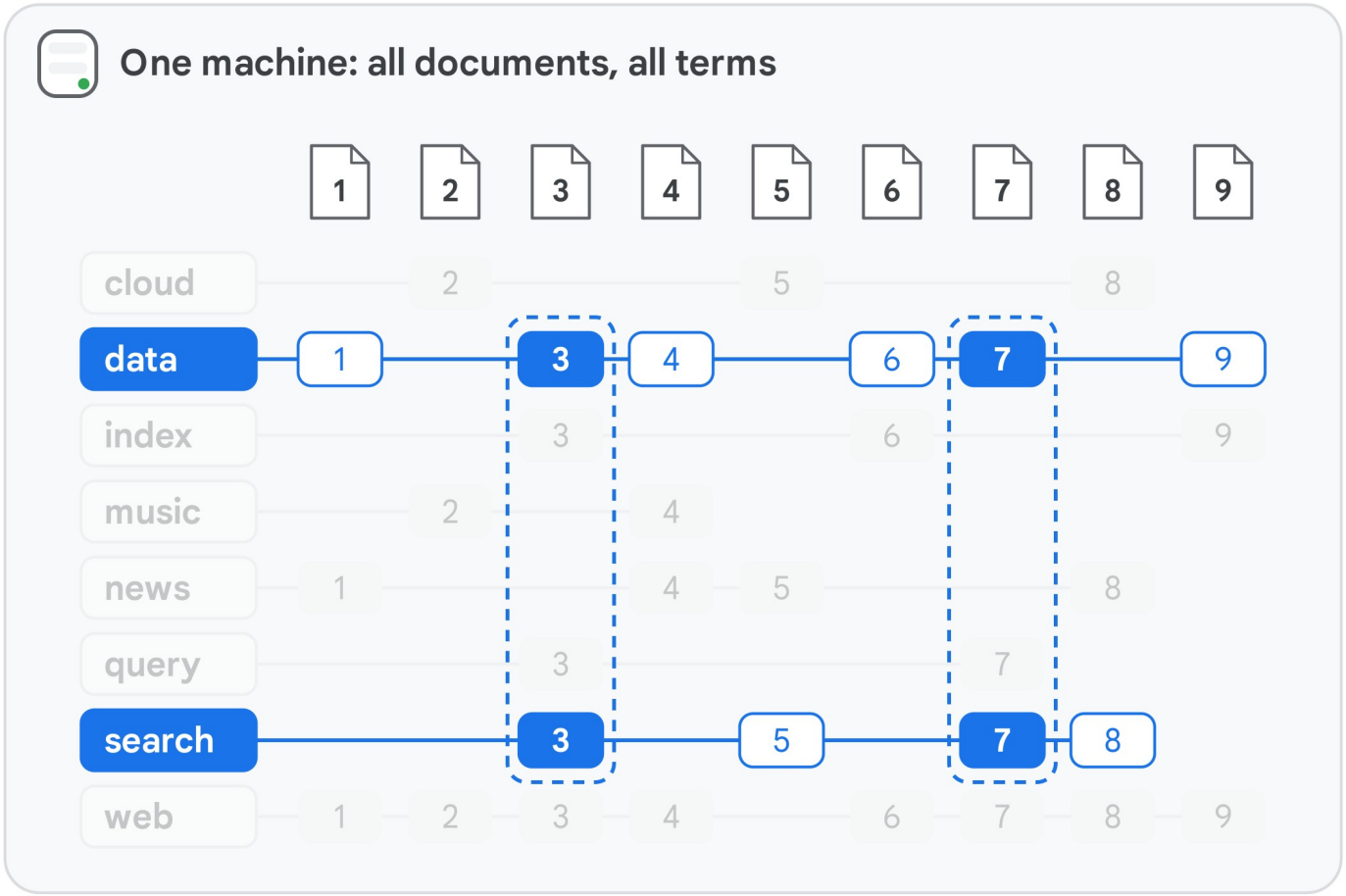


A recap of theory session 5.

One machine: the inverted index

Marc M

Problem | Principles | Example | Trends | More



data search

1 **Look up**
the two postings lists

2 **Intersect**
documents in both lists

3 7

3 **Rank** *illustrative scores*
score them and sort

doc 7	2.1
doc 3	1.4

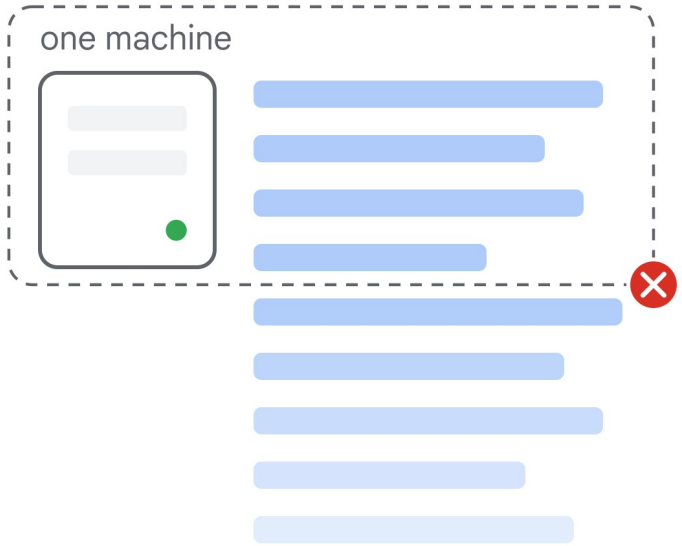
This works well, as long as everything fits on one machine.

Why one machine is not enough for the web

Marc M

Problem | Principles | Example | Trends | More

1 Capacity



index bigger than one machine

2 Speed

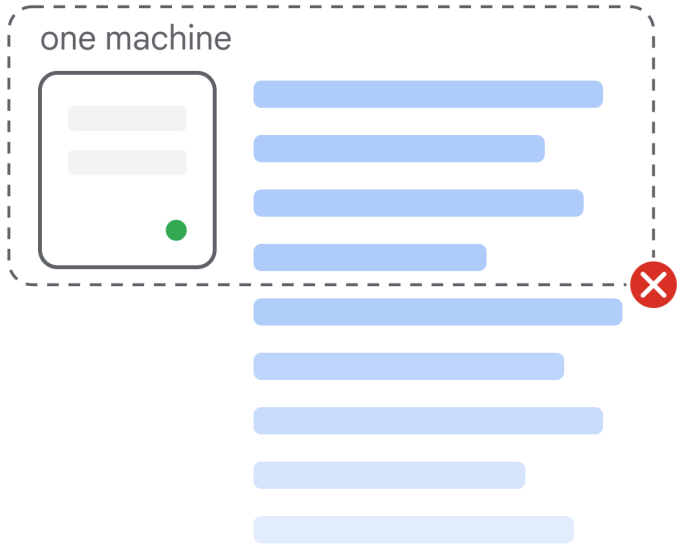
3 Availability

Why one machine is not enough for the web

Marc M

Problem | Principles | Example | Trends | More

1 Capacity



index bigger than one machine

2 Speed



> 0.1 s

millions of candidates per query

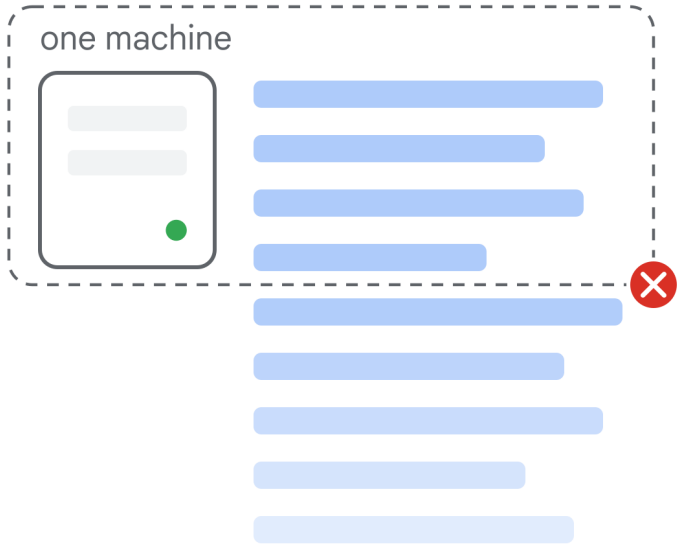
3 Availability

Why one machine is not enough for the web

Marc M

Problem | Principles | Example | Trends | More

1 Capacity



index bigger than one machine

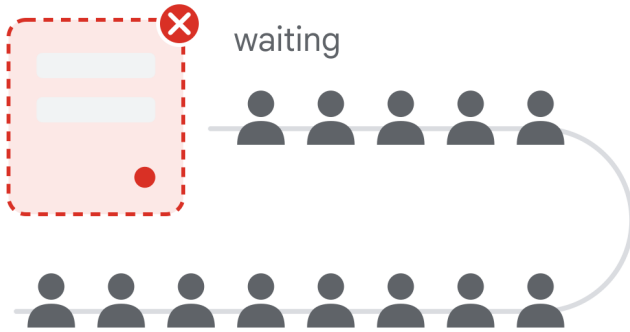
2 Speed



> 0.1 s

millions of candidates per query

3 Availability



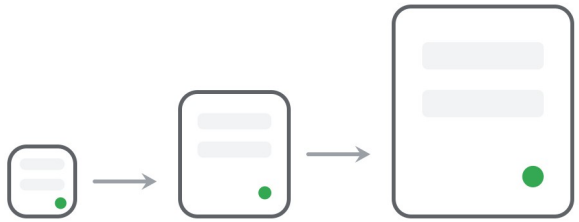
one failure stops everyone

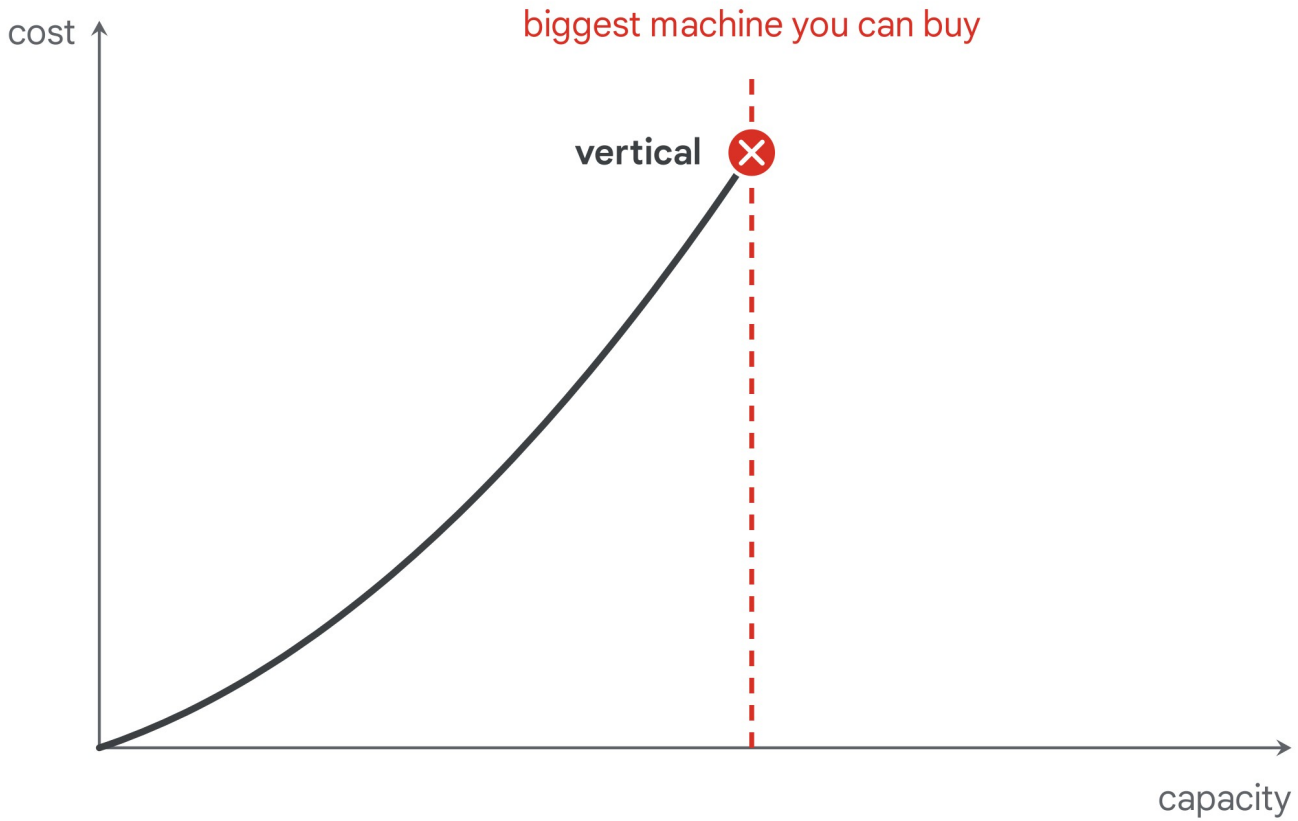
A bigger machine, or more machines?

Marc M

Problem | Principles | Example | Trends | More

Vertical: a bigger machine



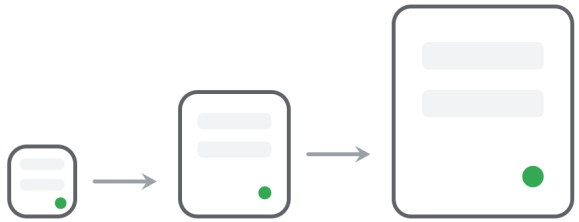


A bigger machine, or more machines?

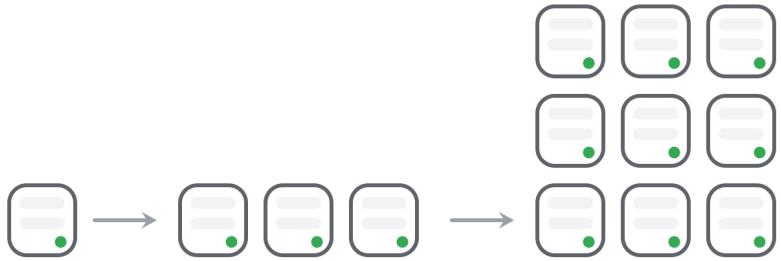
Marc M

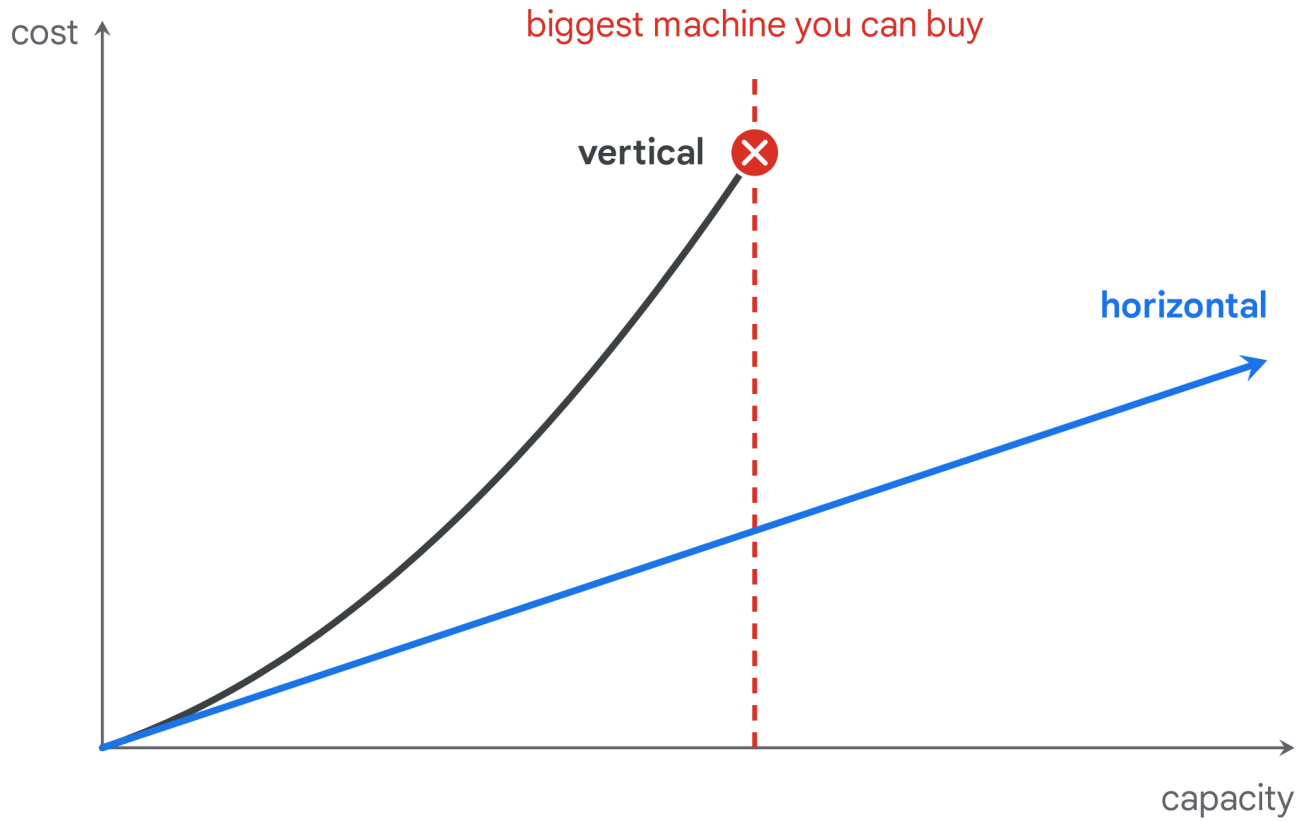
Problem | Principles | Example | Trends | More

Vertical: a bigger machine



Horizontal: more machines





Horizontal scaling: when the web grows, add commodity machines.

From a prototype to millions of machines

Marc M

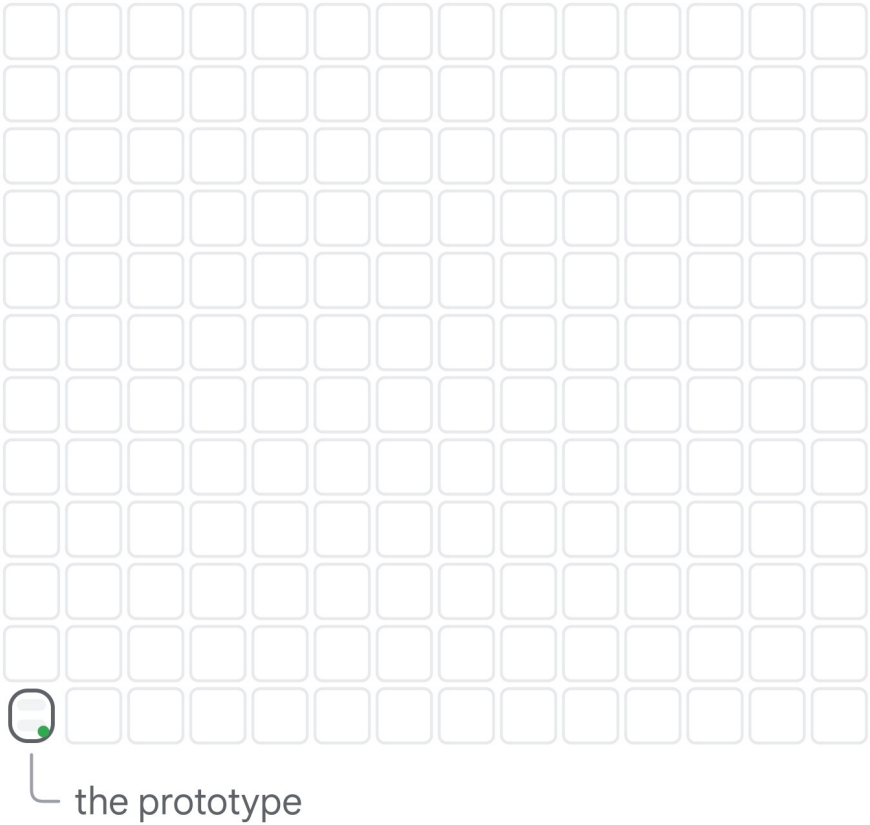
Problem | Principles | Example | Trends | More

24 million

web pages

indexed by a research prototype

1998 — 2003 — 2016



Brin & Page (1998)

From a prototype to millions of machines

Marc M

Problem | Principles | Example | Trends | More

15,000+

commodity PCs

cheap machines, not big servers

1998 — **2003** — 2016



one square = **15,000 machines** (Google in 2003)

Barroso, Dean & Hölzle (2003)

From a prototype to millions of machines

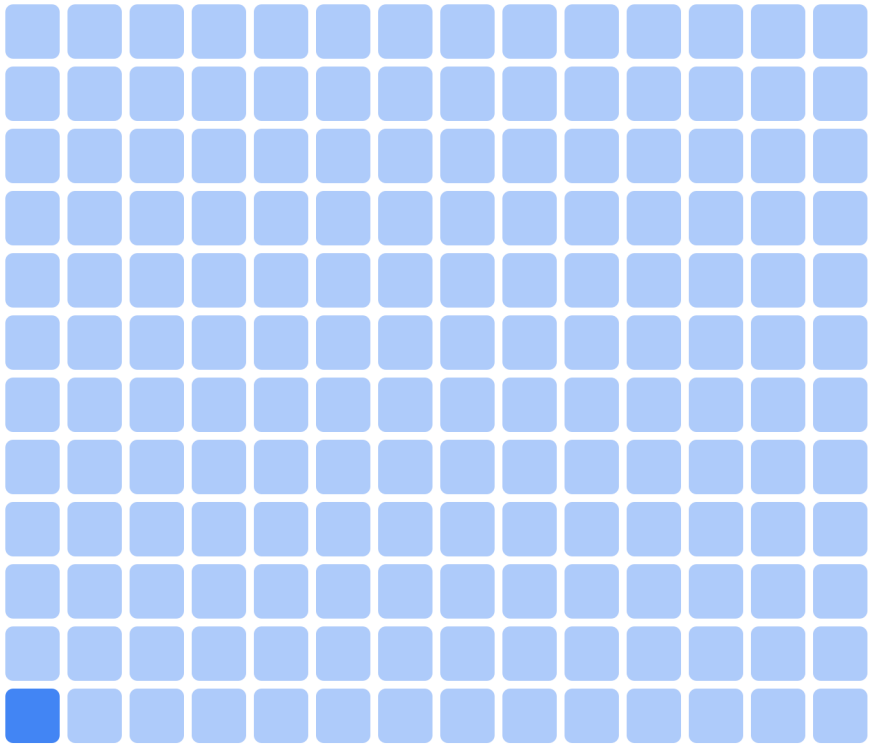
Marc M

Problem | Principles | Example | Trends | More

≈ 2.5 million

servers, Gartner estimate (July 2016)

Google publishes no official server count



one square = **15,000 machines** (Google in 2003)

1998 — 2003 — **2016**

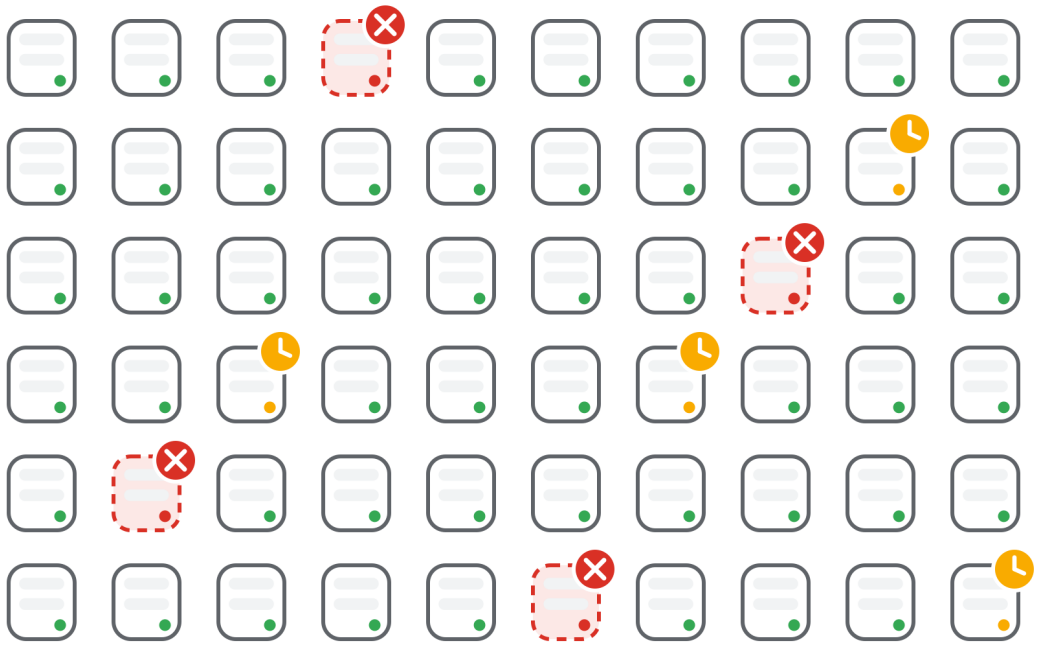
Brin & Page (1998), Barroso, Dean & Hölzle (2003), Gartner (2016) as cited in IRWA theory session 5

Many unreliable machines, one reliable engine

Marc M

Problem | Principles | Example | Trends | More

Millions of cheap machines



something is always failing or slow

how?

One search engine

search the web

- page 1
- page 2
- page 3

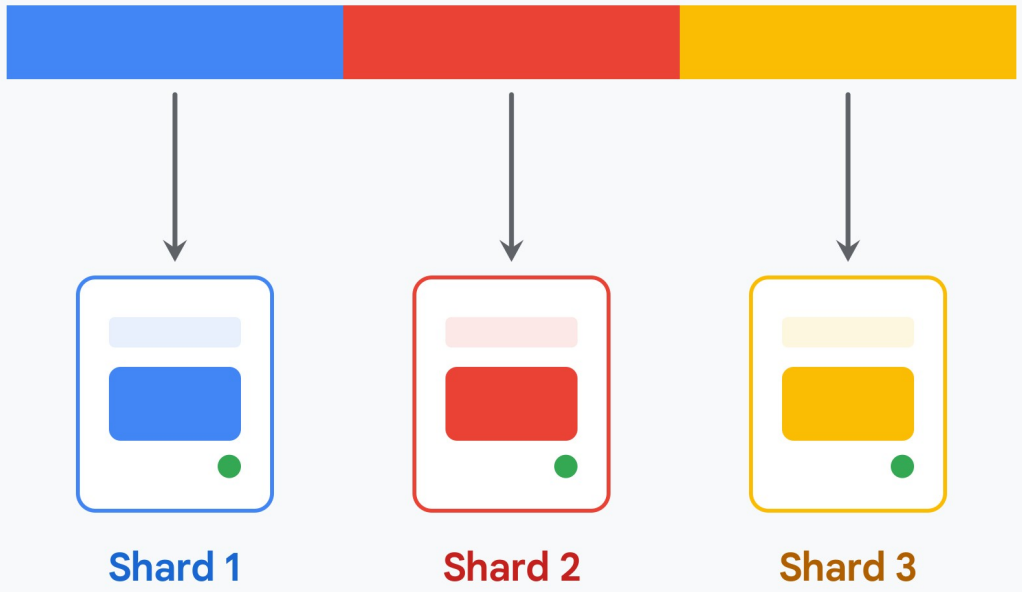
one fast, reliable answer

Two tools: sharding and replication

Marc M

Problem | Principles | Example | Trends | More

Sharding: split the data



each machine holds a different piece

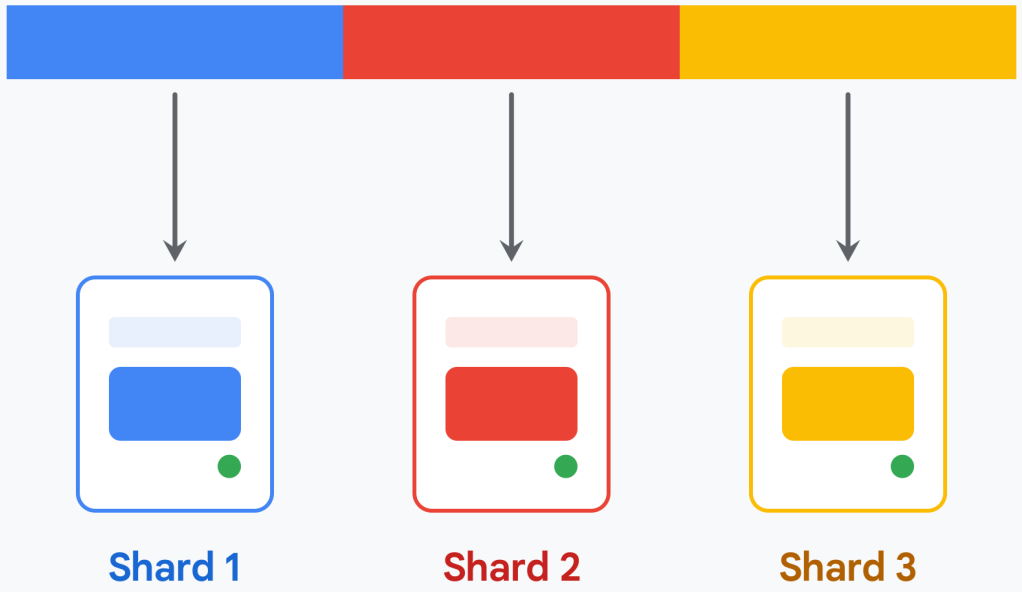
Replication: keep copies

Two tools: sharding and replication

Marc M

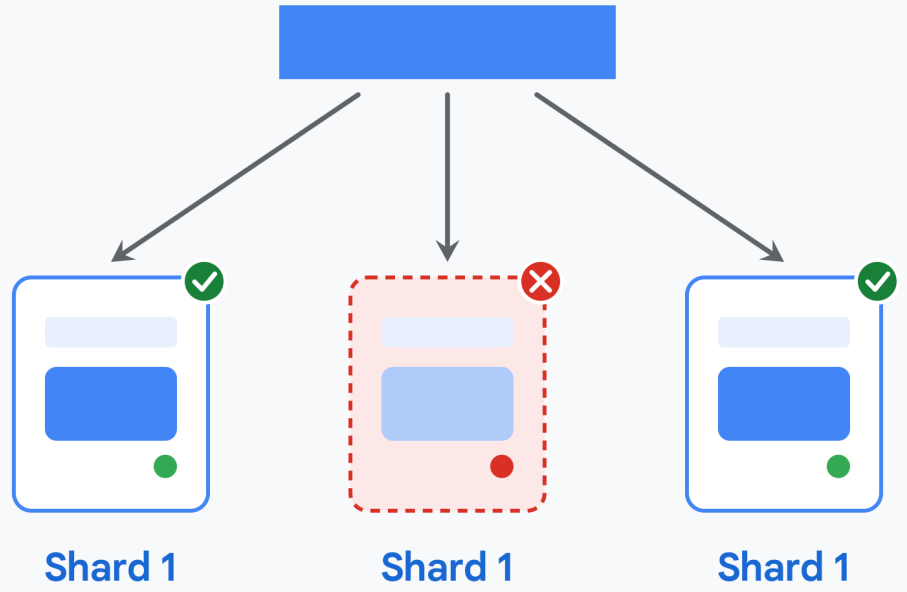
Problem | Principles | Example | Trends | More

Sharding: split the data



each machine holds a different piece

Replication: keep copies



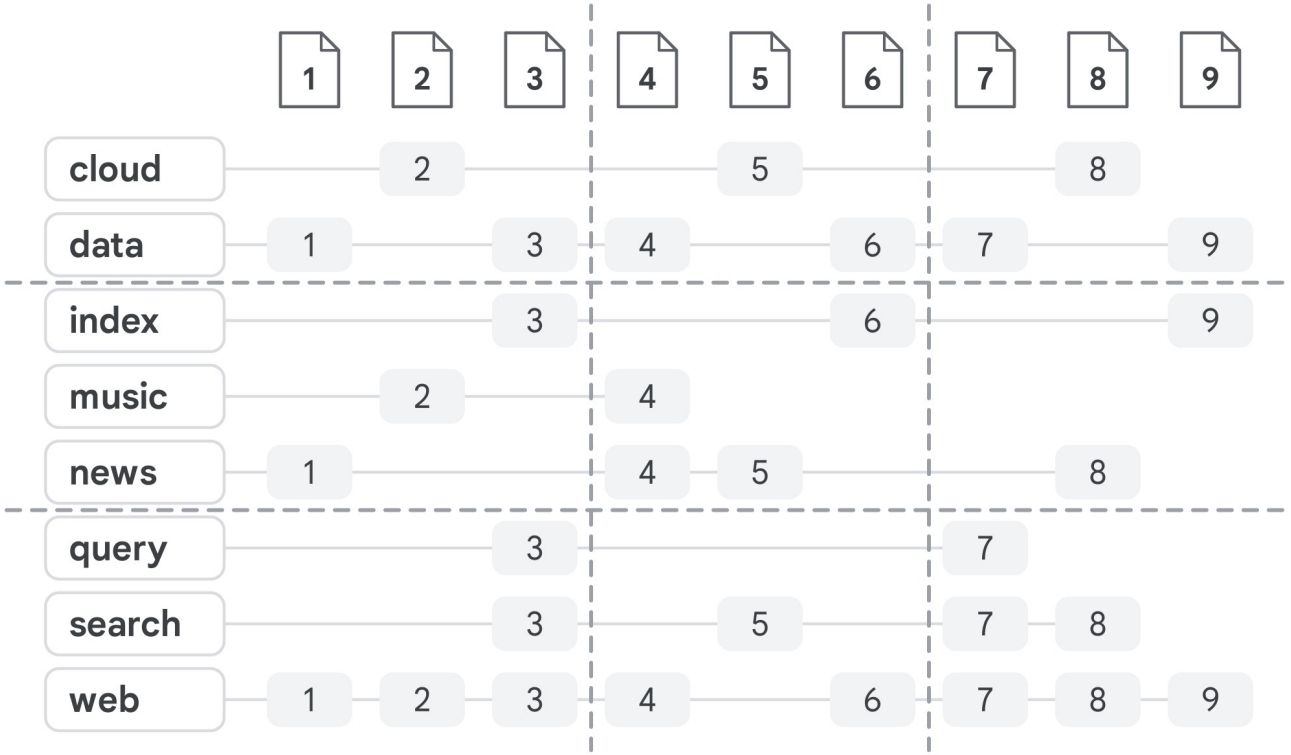
a machine can die, a copy answers

Specific to search: how do we split an inverted index and still return one ranked list?

How do we split an inverted index?

Marc M

Problem | **Principles** | Example | Trends | More



Three machines.

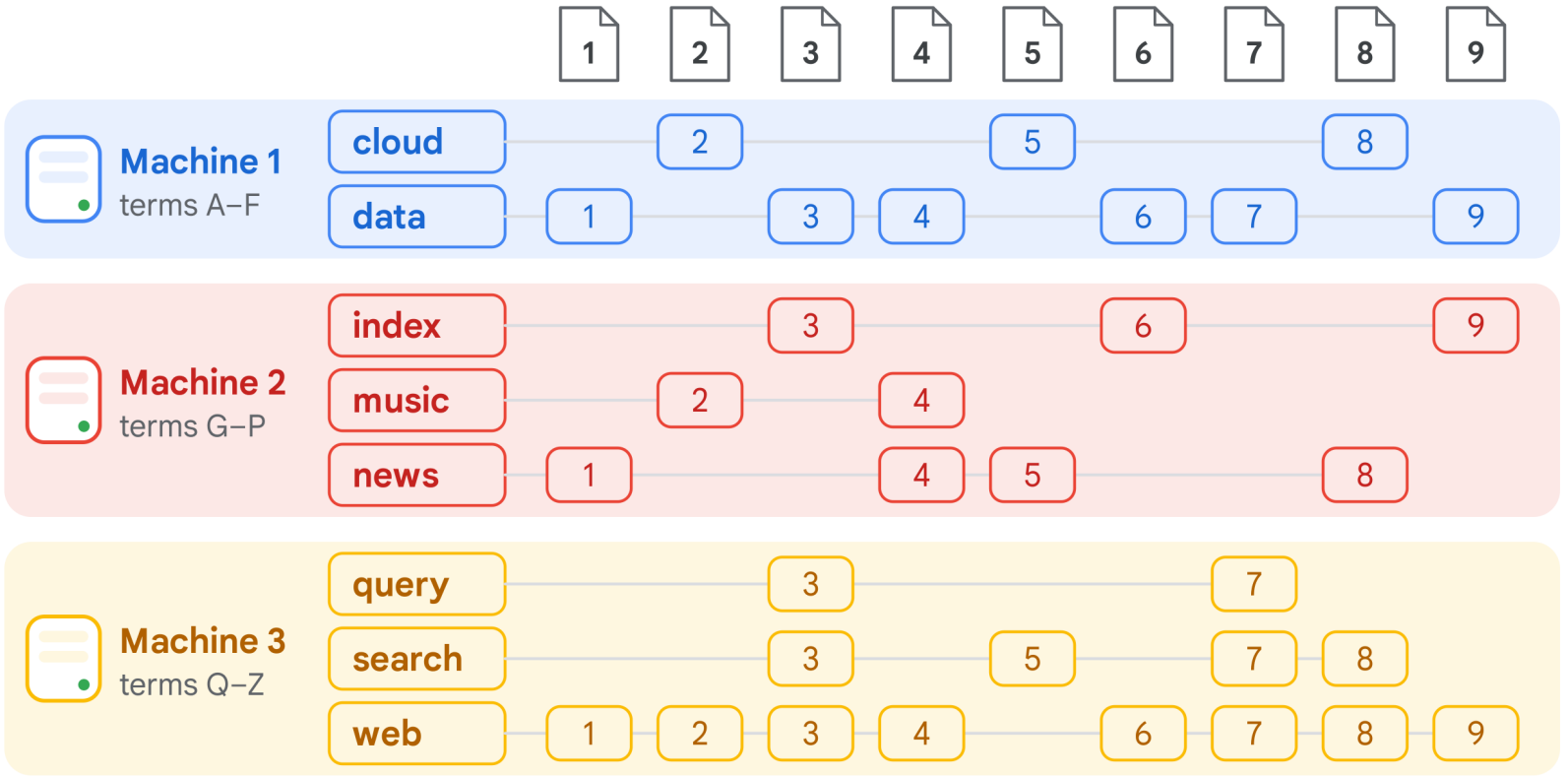
Where does each part go?



Cut by rows: terms

Cut by columns: documents

Option 1: split by term



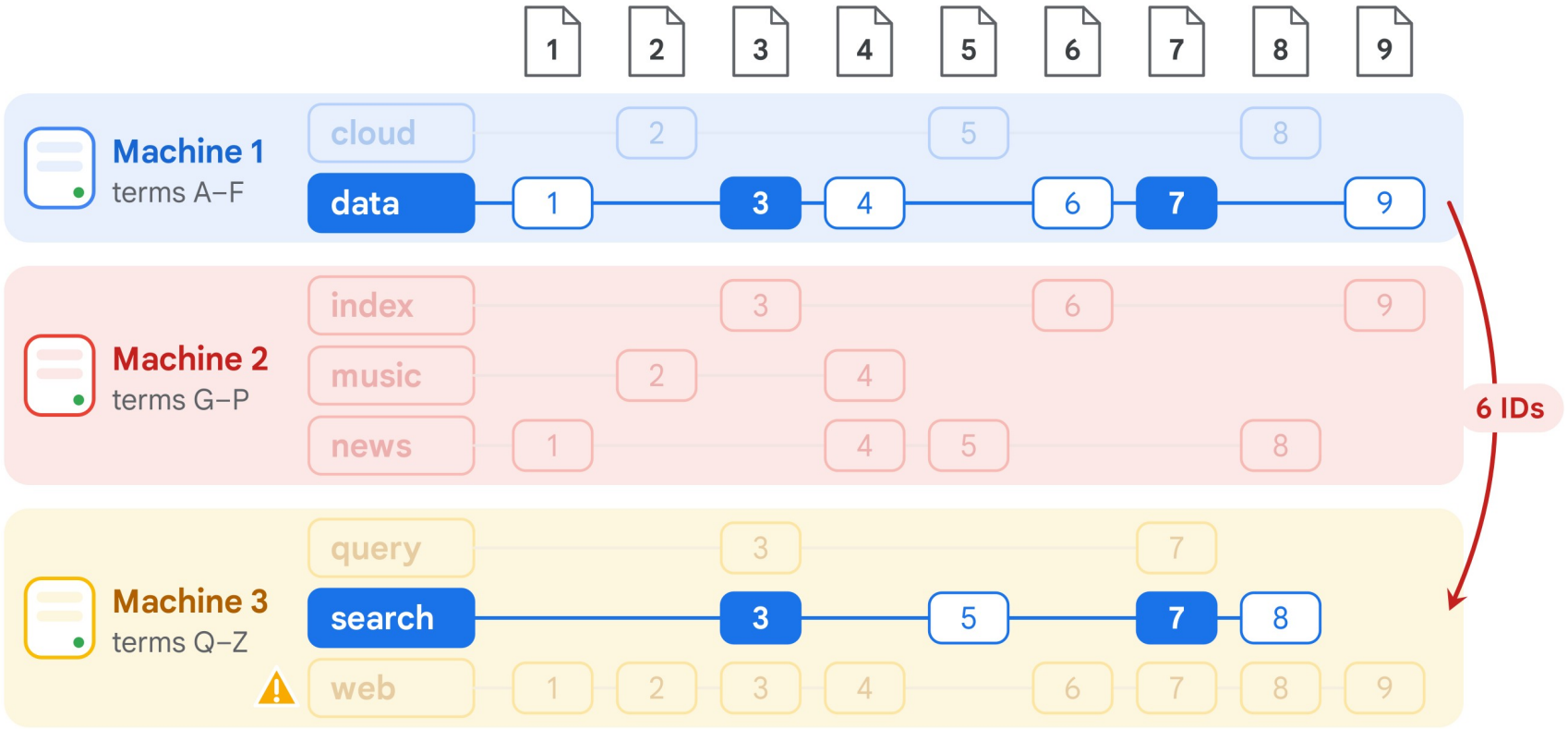
Term partitioning
Each machine owns the whole postings lists of some terms.

A query only visits the machines that own its terms

Option 1: split by term

Marc M

Problem | **Principles** | Example | Trends | More



Lists cross the network
to intersect, one machine ships a whole list to the other

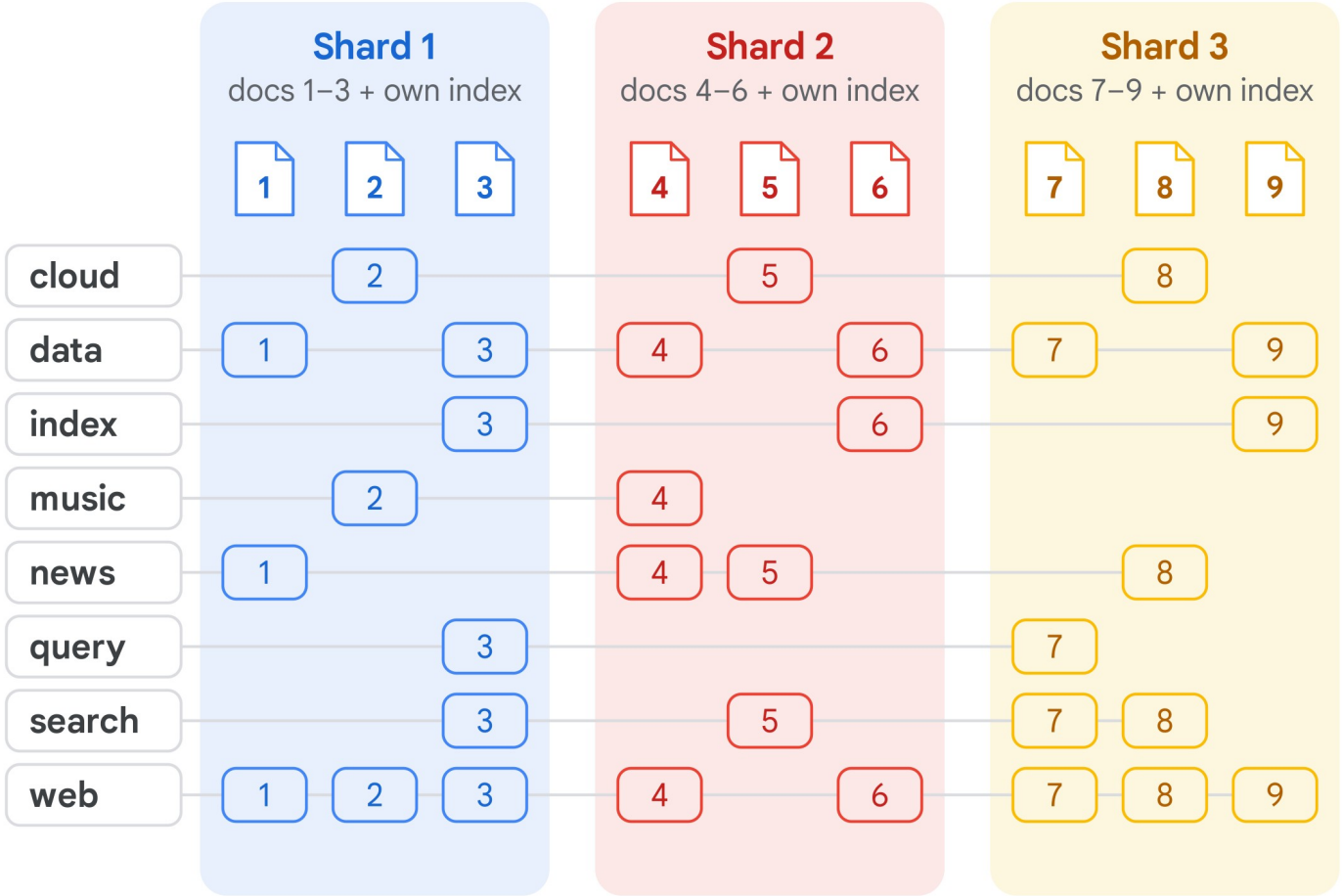
Hot terms
"web" is in 8 of 9 documents and many queries: machine 3 does most of the work

Updates are spread
one new page touches one machine per term in it

Option 2: split by document

Marc M

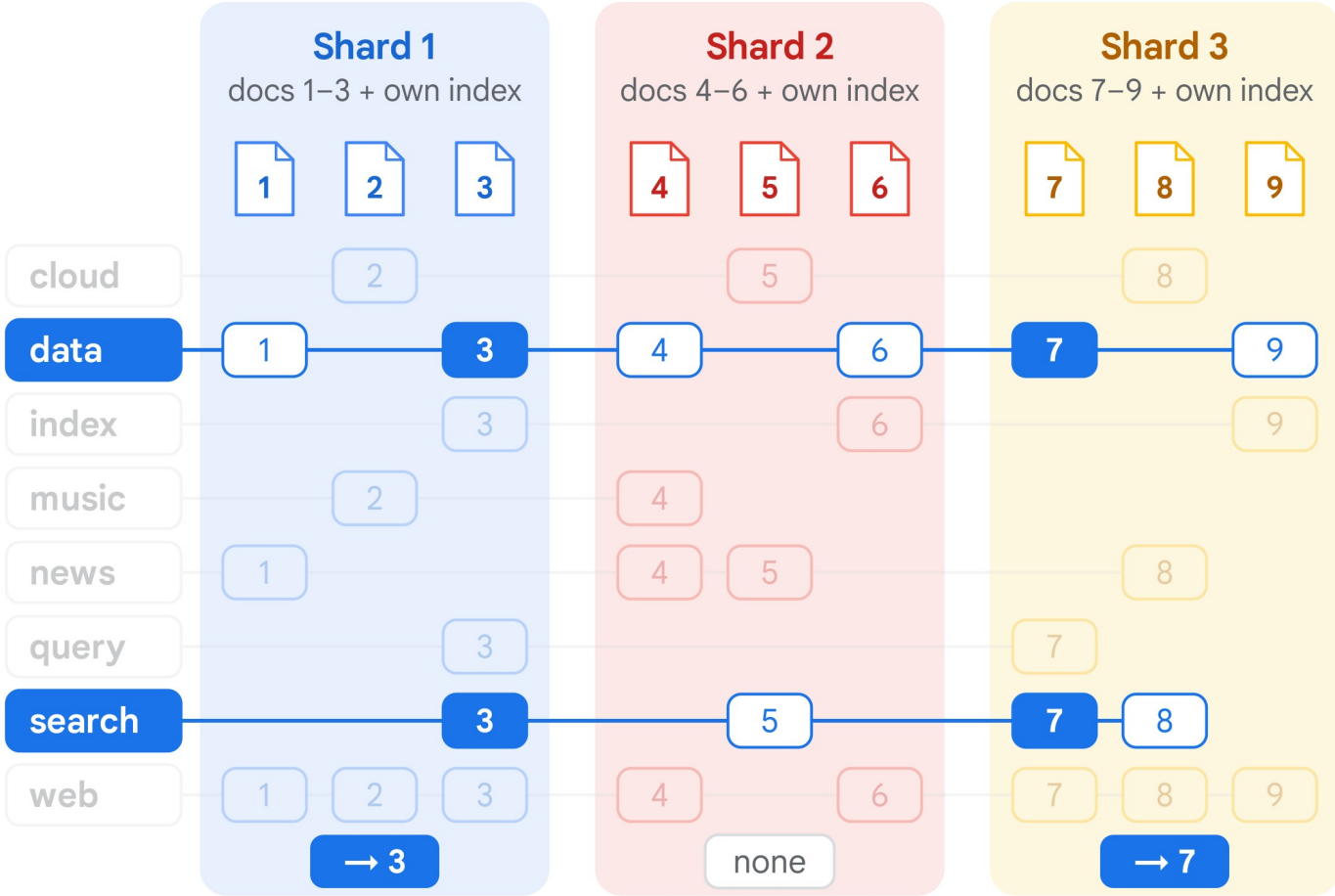
Problem | **Principles** | Example | Trends | More



Document partitioning
Each shard owns some documents and a complete, small index over them.

Every query
goes to every shard

Option 2: split by document



All work is local
lookup, intersection and scoring inside each shard

Small answers travel
only each shard's local top results

Easy updates
a new page touches one shard

Losing a shard
loses a slice of results, not whole terms

Term or document partitioning?

Marc M

Problem | **Principles** | Example | Trends | More

	Term partitioning	Document partitioning
Machines per query	✅ only the owners of its terms	
Network traffic	❌ whole postings lists	
Adding a page	❌ one machine per term	
Load	❌ hot terms overload a machine	
Losing a machine	❌ some terms disappear	

Document partitioning wins in practice

Marc M

Problem | **Principles** | Example | Trends | More

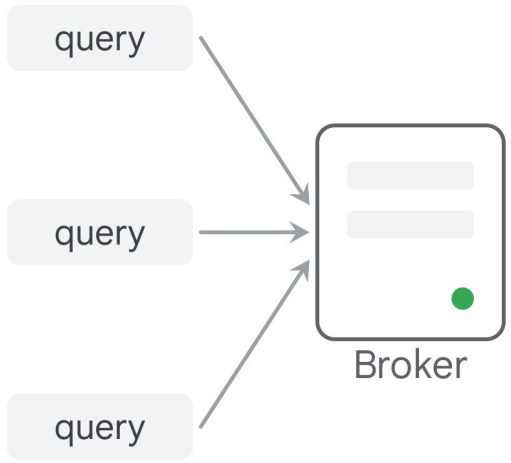
	Term partitioning	Document partitioning (preferred)
Machines per query	✓ only the owners of its terms	✗ every shard
Network traffic	✗ whole postings lists	✓ small top-k lists
Adding a page	✗ one machine per term	✓ one shard
Load	✗ hot terms overload a machine	✓ spread evenly
Losing a machine	✗ some terms disappear	✓ a slice of the results

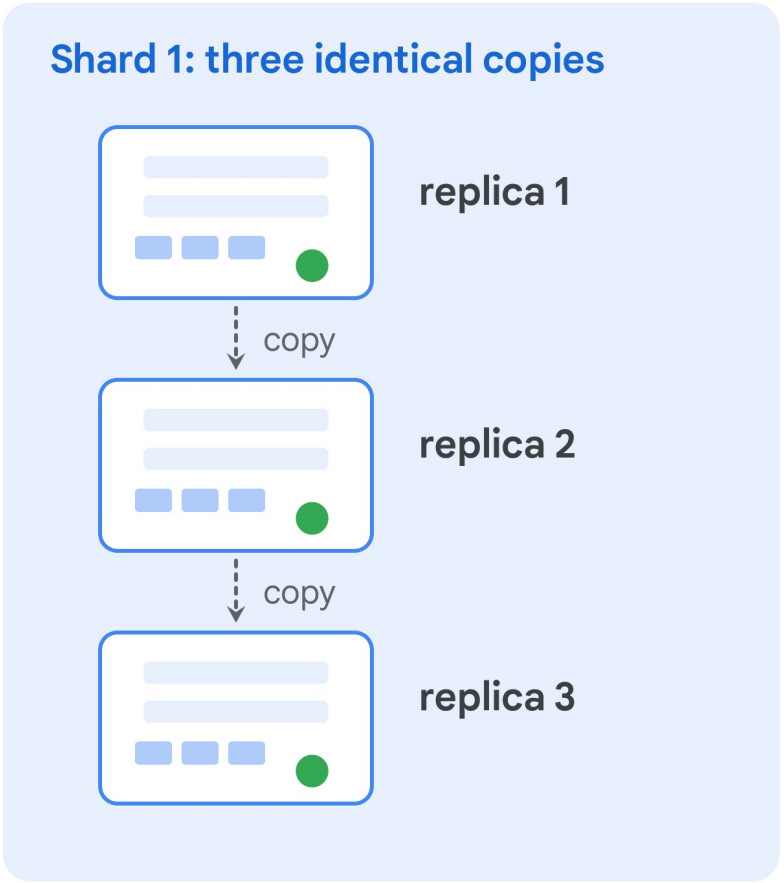
From now on, a **shard** is a slice of the documents with its own full index. Each document is in exactly one shard.

Google in 2003: each shard held a random subset of the documents (Barroso, Dean & Hölzle)

Moffat, Webber, Zobel & Baeza-Yates (2007), IRWA theory session 5

Replication: two jobs, not one





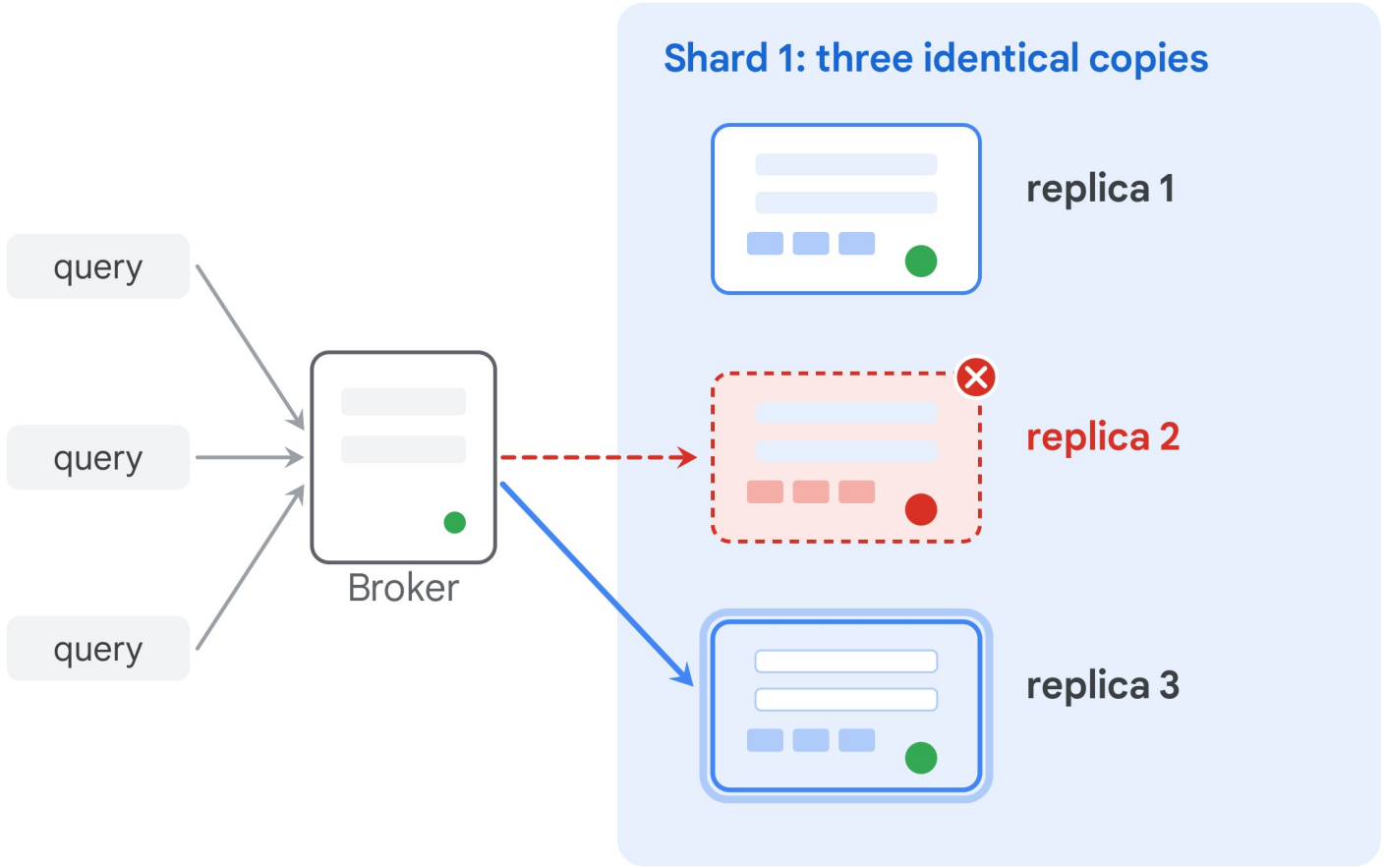
1 Availability
If a replica dies, the broker sends the query to another copy.
The user never notices.

2 Throughput
One replica answers only so many queries per second.
3 replicas ≈ 3× the throughput.

Replication: two jobs, not one

Èric

Problem | **Principles** | Example | Trends | More



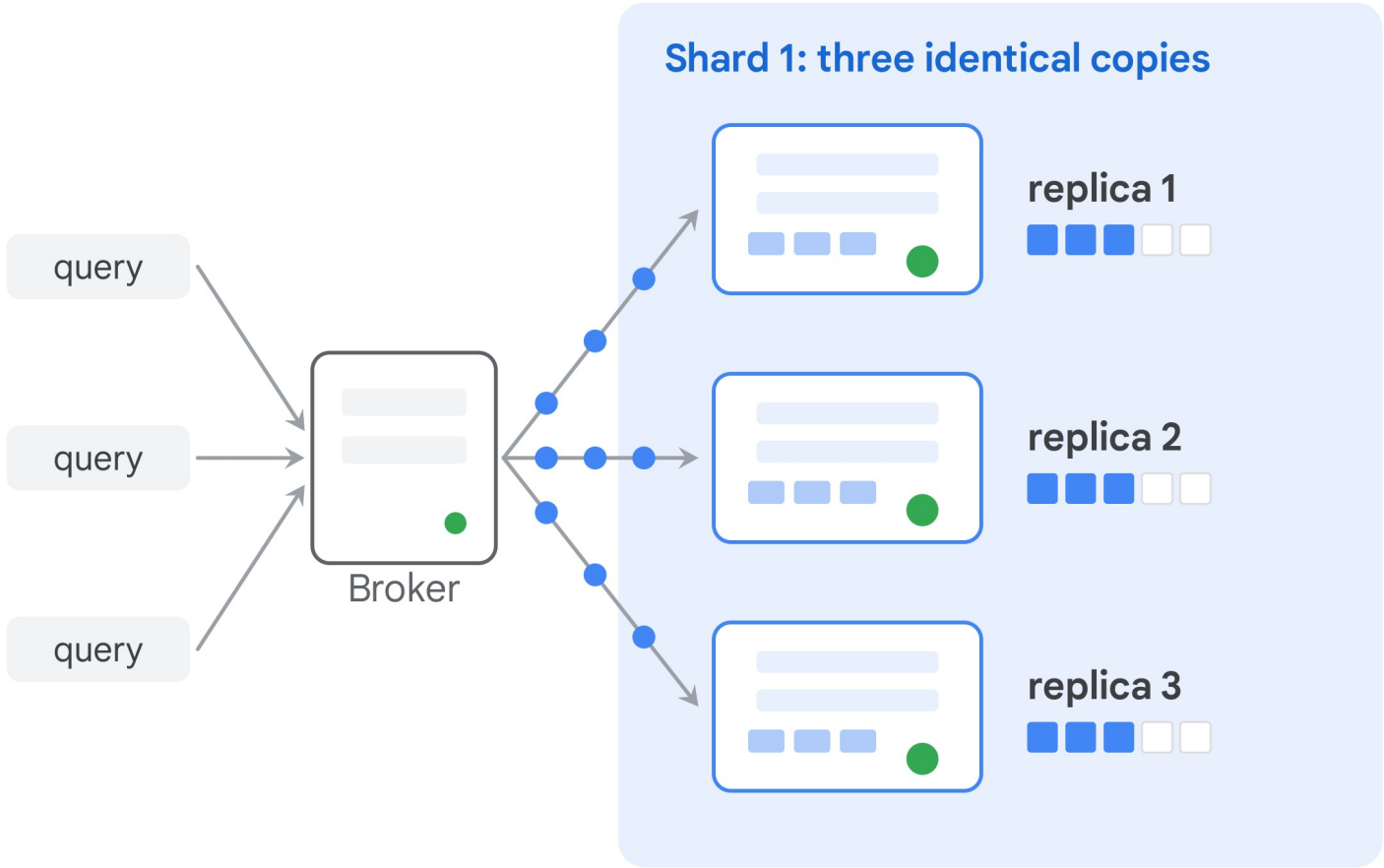
1 Availability
If a replica dies, the broker sends the query to another copy.
The user never notices.

2 Throughput
One replica answers only so many queries per second.
3 replicas ≈ 3× the throughput.

Replication: two jobs, not one

Èric

Problem | **Principles** | Example | Trends | More



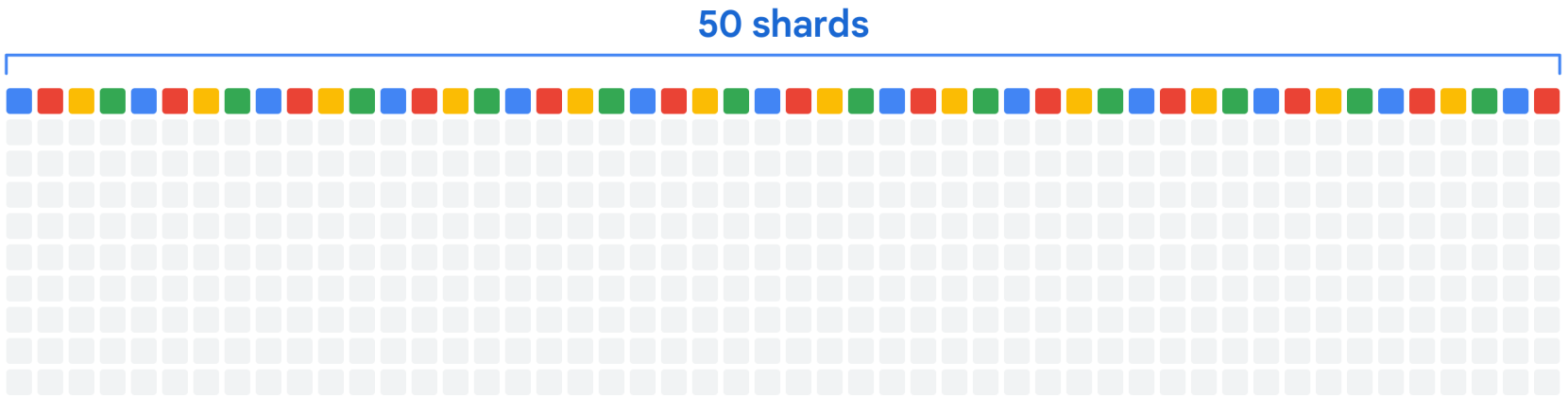
1 **Availability**
If a replica dies, the broker sends the query to another copy.
The user never notices.

2 **Throughput**
One replica answers only so many queries per second.
3 replicas ≈ 3× the throughput.

Shards for data, replicas for queries

Èric È

Problem | **Principles** | Example | Trends | More



one machine, 128 GB

1 copy of the index

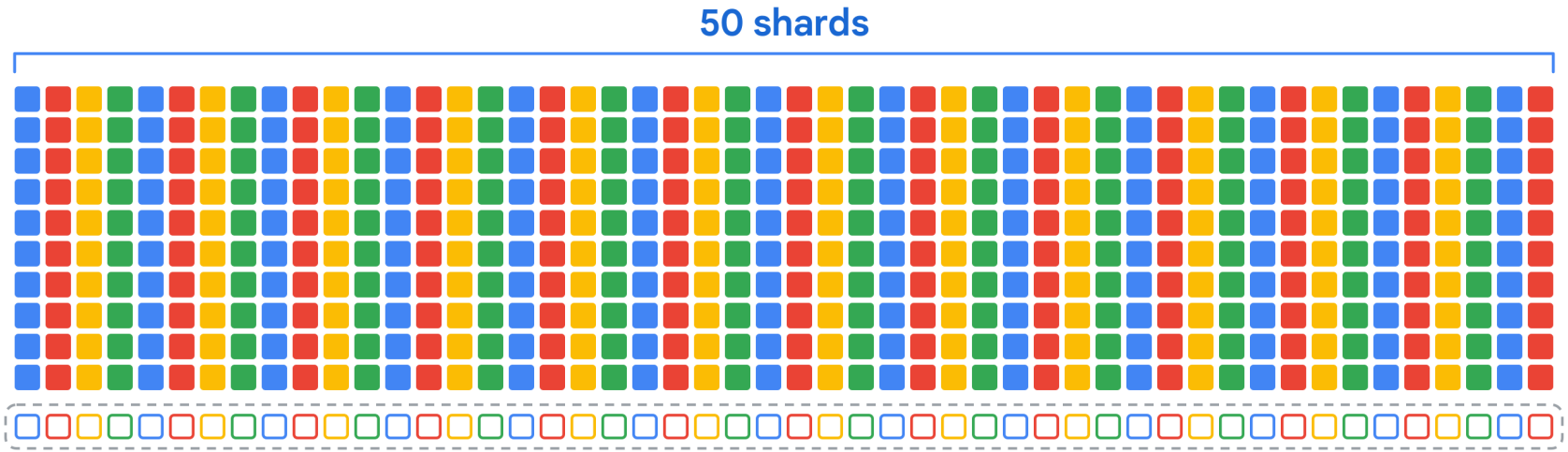
6 TB ÷ 128 GB per machine ≈ **50 shards**

illustrative numbers

Shards for data, replicas for queries

Èric

Problem | **Principles** | Example | Trends | More



one machine, 128 GB

1 copy of the index

10 replicas

1 more row: +500 queries/s and costs +50 machines

spare: a failed machine does not cut peak capacity

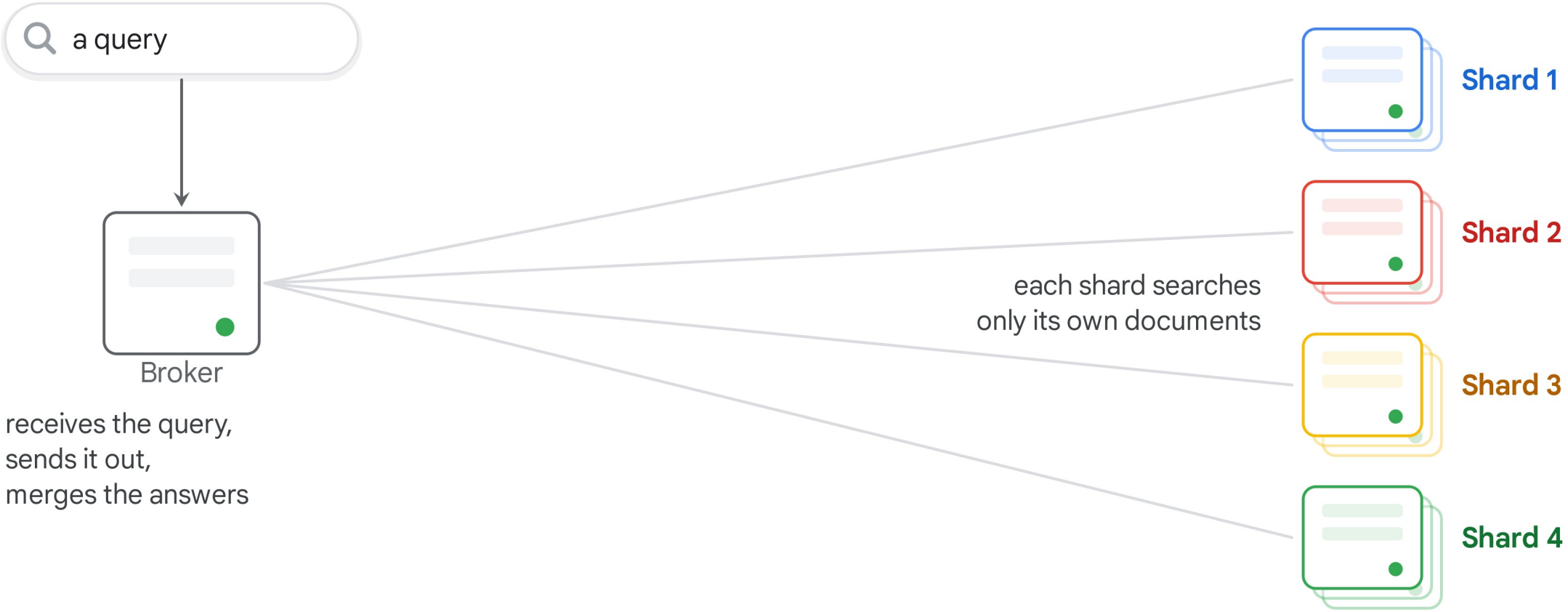
6 TB ÷ 128 GB per machine ≈ **50 shards**

5,000 queries/s ÷ 500 per copy = 10 replicas, + 1 spare = **11 replicas**

50 × 11 = 550 machines

illustrative numbers

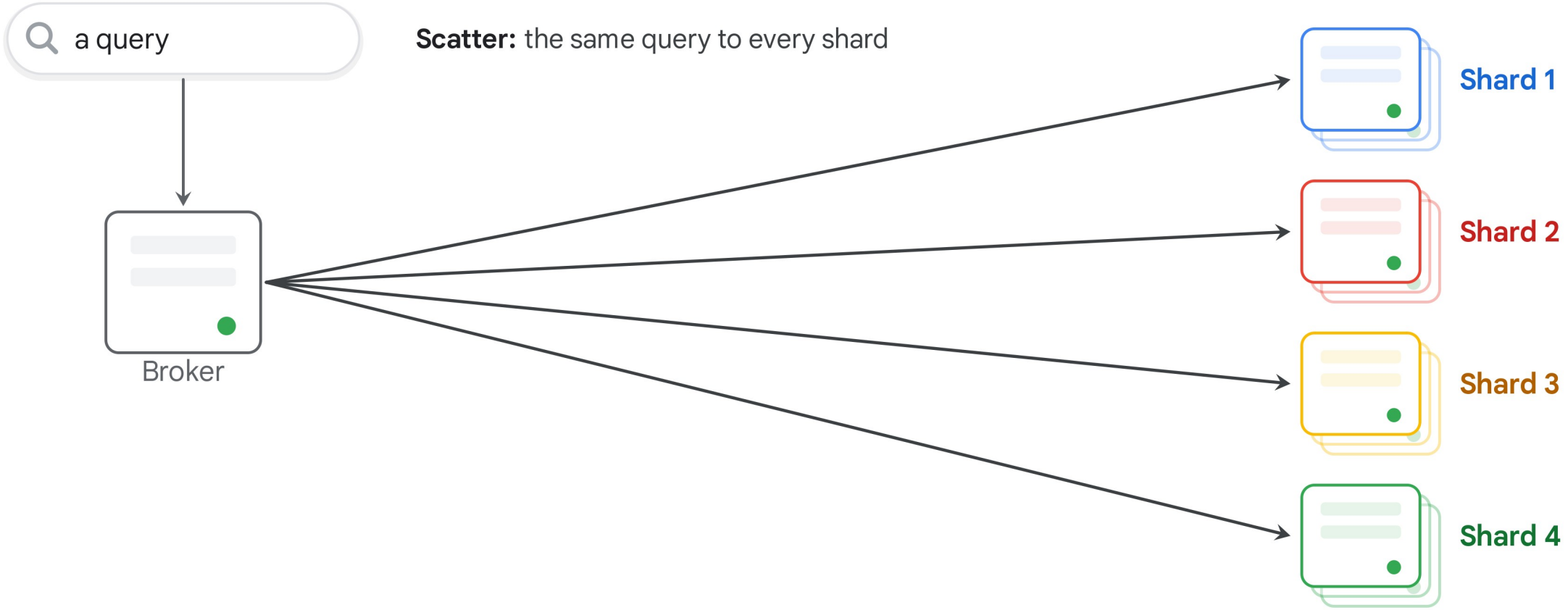
Shards and a broker: scatter-gather



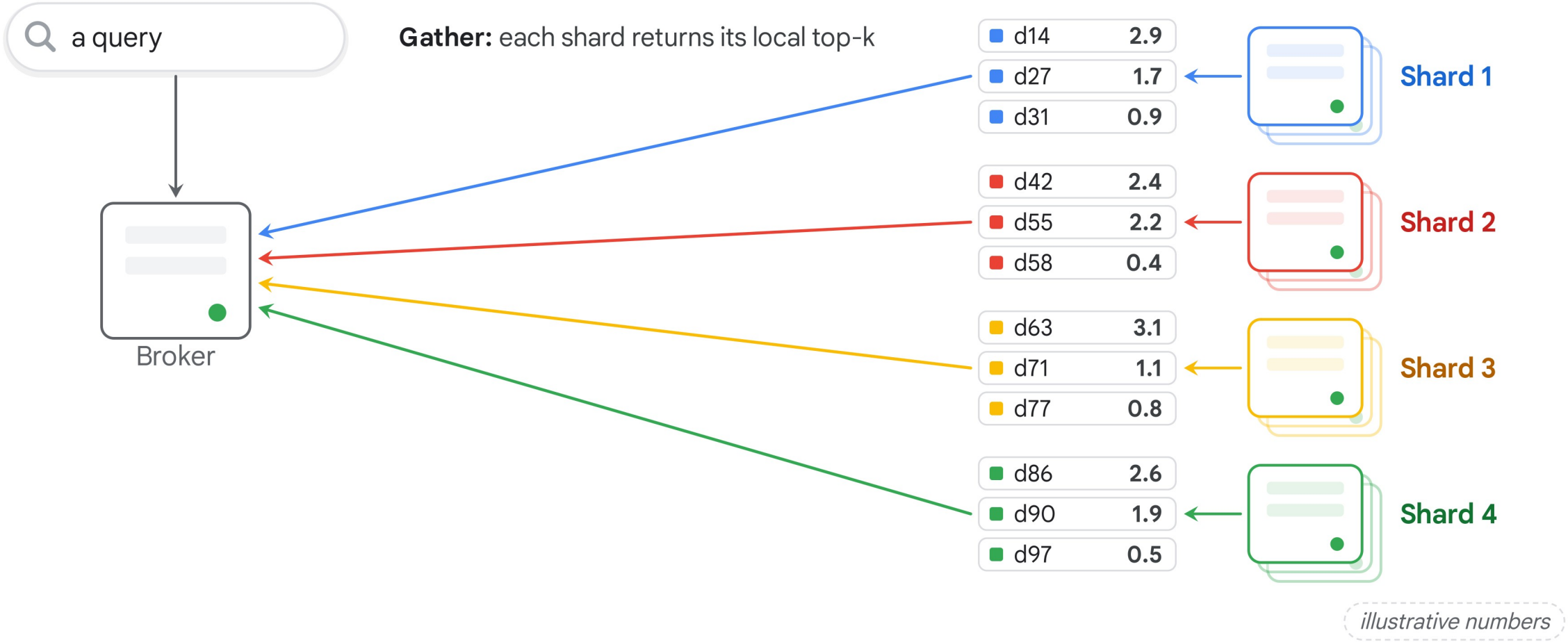
Shards and a broker: scatter-gather

Èric

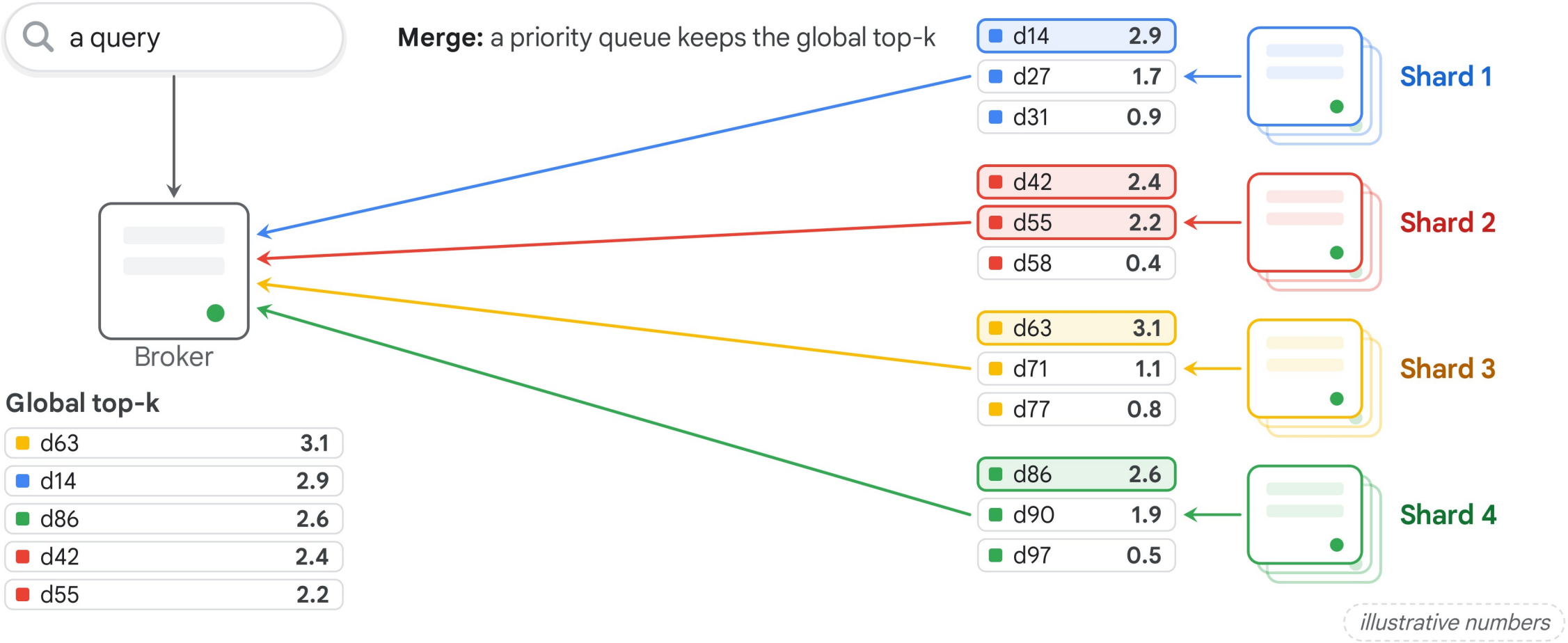
Problem | **Principles** | Example | Trends | More



Shards and a broker: scatter-gather



Shards and a broker: scatter-gather



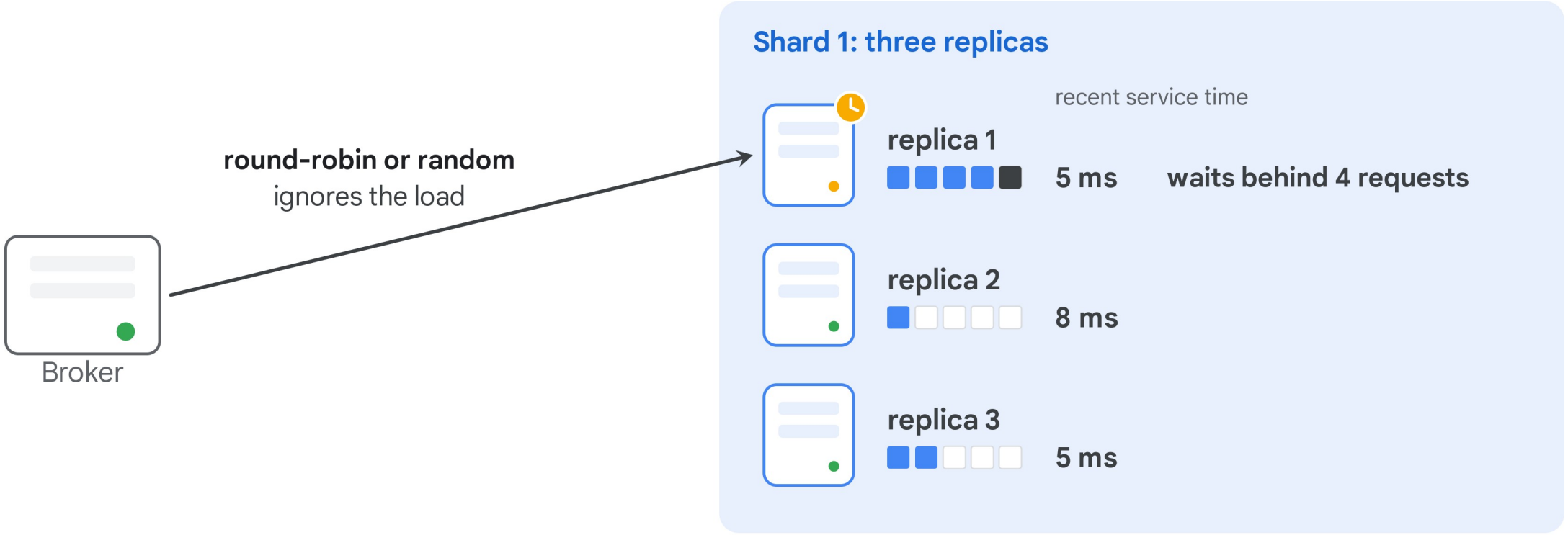
Decision 1: which replica answers?

Èric

Problem | **Principles** | Example | Trends | More

1 **Which replica?** 2 What travels back? 3 Which shards?

illustrative numbers



Simplified from C3 (Suresh et al. 2015), which Elasticsearch's adaptive replica selection implements

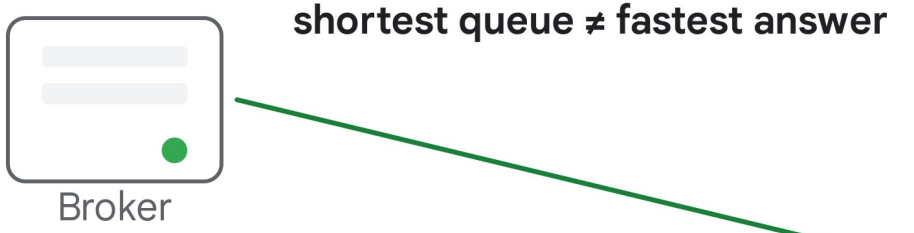
Decision 1: which replica answers?

1 **Which replica?** 2 What travels back? 3 Which shards?

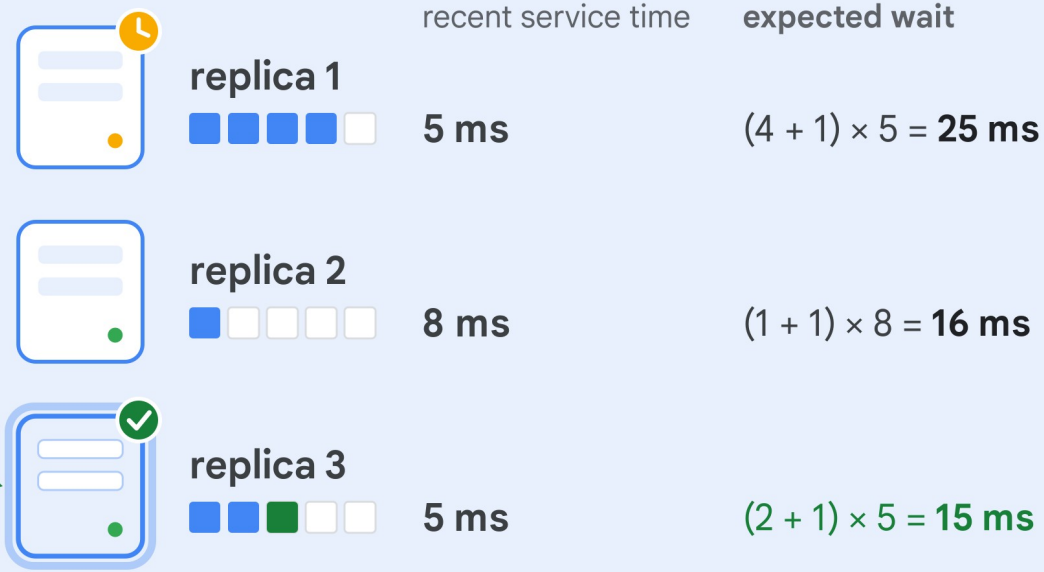
illustrative numbers

least expected wait
(queue + 1) × recent service time

shortest queue ≠ fastest answer



Shard 1: three replicas



Simplified from C3 (Suresh et al. 2015), which Elasticsearch's adaptive replica selection implements

Decision 2: what travels back?

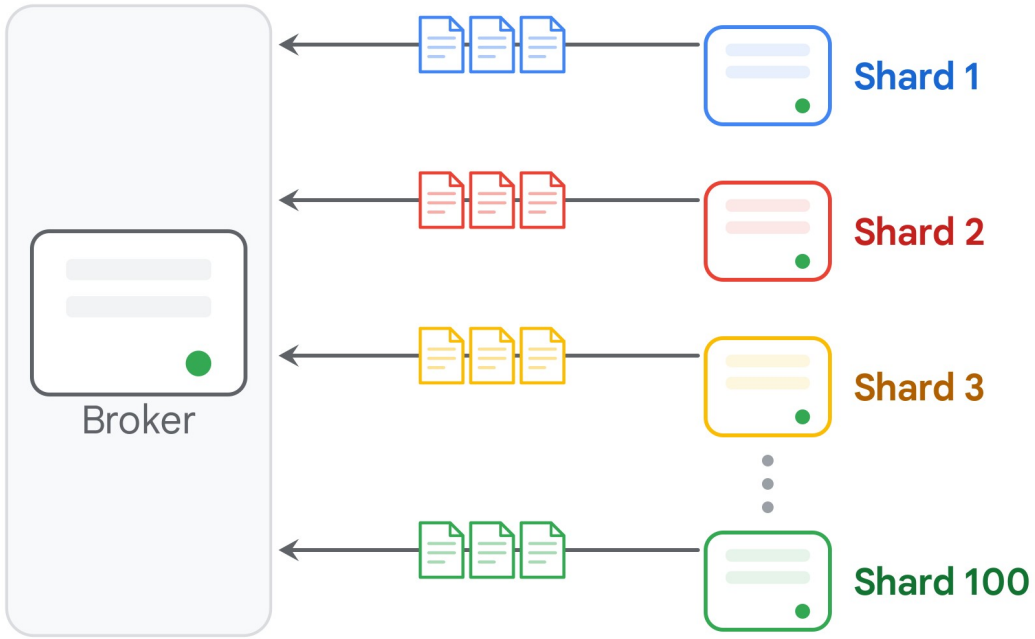
Èric

Problem | **Principles** | Example | Trends | More

1 Which replica? | 2 **What travels back?** | 3 Which shards?

illustrative numbers

← **One phase:** every shard sends its top 10 with snippets



Bytes back to the broker, per query

One phase

2 MB

100 shards × 10 hits × 2 KB

99% thrown away



Decision 2: what travels back?

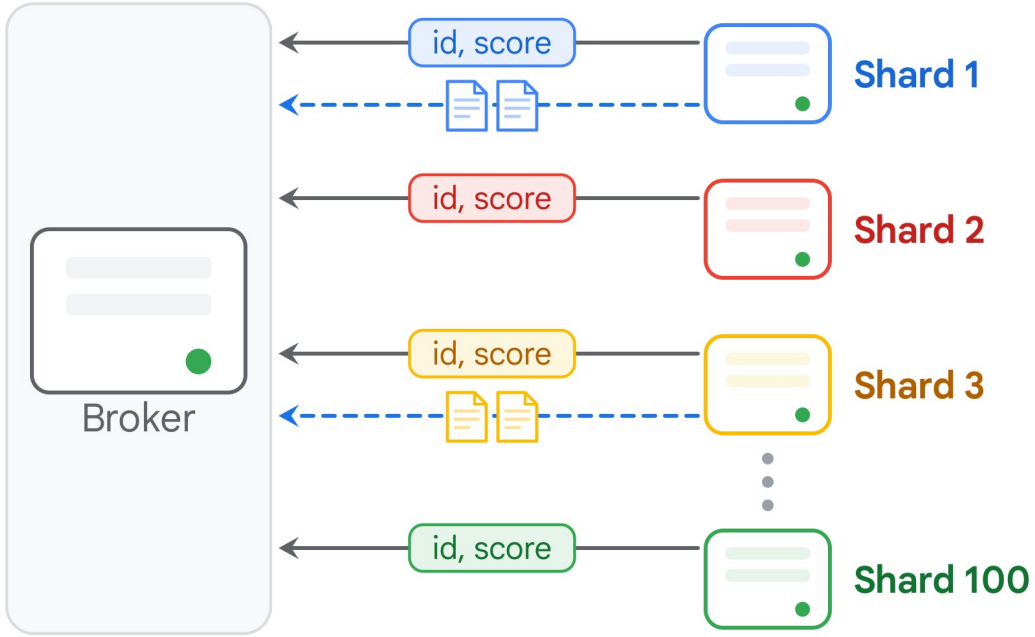
Èric

1 Which replica? | 2 **What travels back?** | 3 Which shards?

illustrative numbers

← **Phase 1:** every shard sends only id and score

<--- **Phase 2:** snippets, only from shards with winners



Bytes back to the broker, per query

One phase

2 MB

100 shards × 10 hits × 2 KB — *99% thrown away*

Two phases

32 KB

12 KB (phase 1) + 20 KB (phase 2)

0 — 1 MB — 2 MB

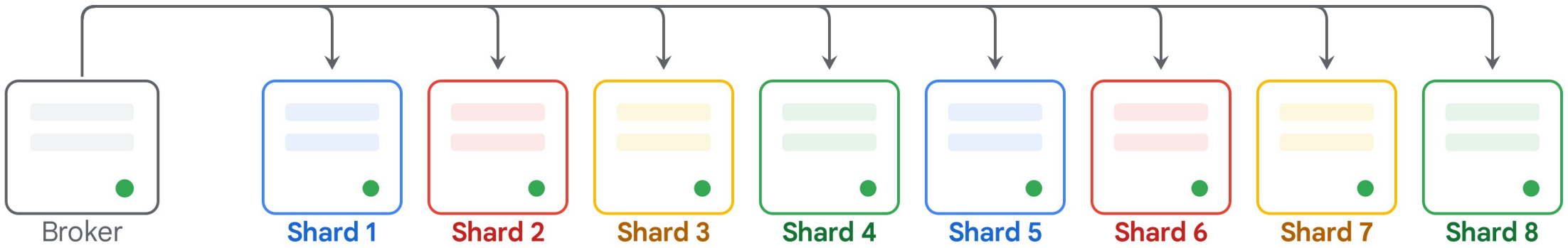
Two phases as in Barroso, Dean & Hölzle (2003): index servers return document IDs, then document servers build titles and snippets

Decision 3: which shards are asked?

Èric

Problem | **Principles** | Example | Trends | More

1 Which replica? 2 What travels back? 3 **Which shards?**



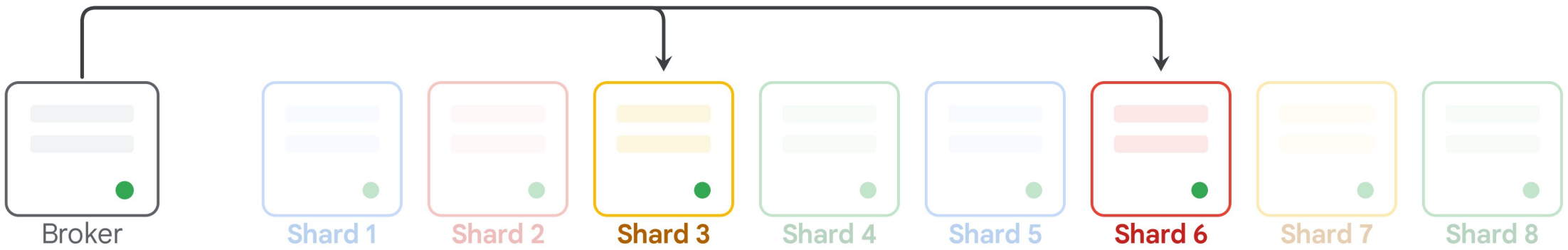
Default: ask every shard, 8 of 8

Decision 3: which shards are asked?

Èric

Problem | **Principles** | Example | Trends | More

1 Which replica? 2 What travels back? **3 Which shards?**



Selective search: ask only some, here 2 of 8

Heavy work where the data lives, only small results travel

The merging trap: local statistics

Èric

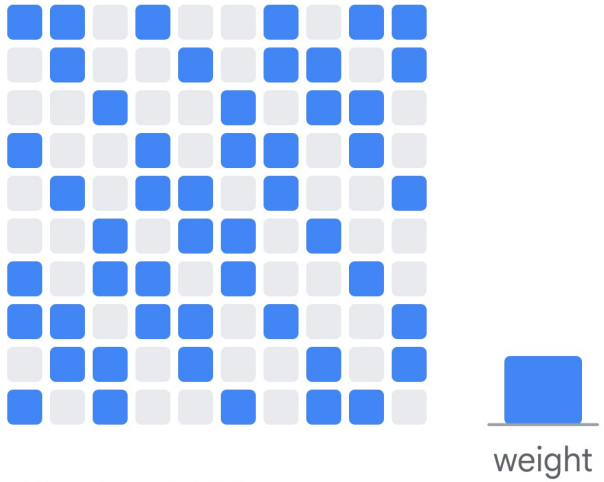
Problem | **Principles** | Example | Trends | More

Each square is one document, coloured if it contains “cloud”

illustrative numbers

Shard 1

100 documents



df = 50 of 100

idf = log(100 / 50) = **0.30**

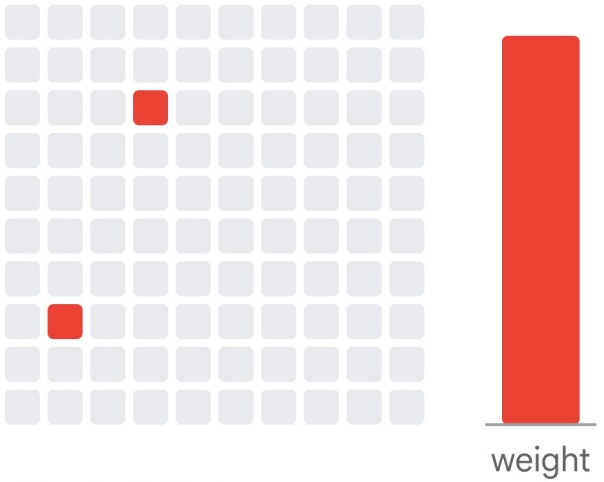
Same kind of match,

≈ 5.7× the weight on Shard 2

Melnik, Raghavan, Yang & Garcia-Molina (2001)

Shard 2

100 documents



df = 2 of 100

idf = log(100 / 2) = **1.70**

The merging trap: fixed by global statistics

Èric È

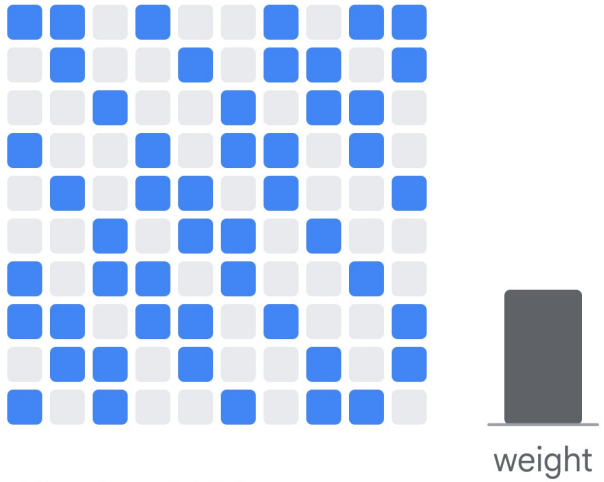
Problem | **Principles** | Example | Trends | More

Each square is one document, coloured if it contains “cloud”

illustrative numbers

Shard 1

100 documents



weight

df = 50 of 100

idf = log(100 / 50) = **0.30**

idf = log(200 / 52) = **0.59** ✓

Global statistics

computed offline

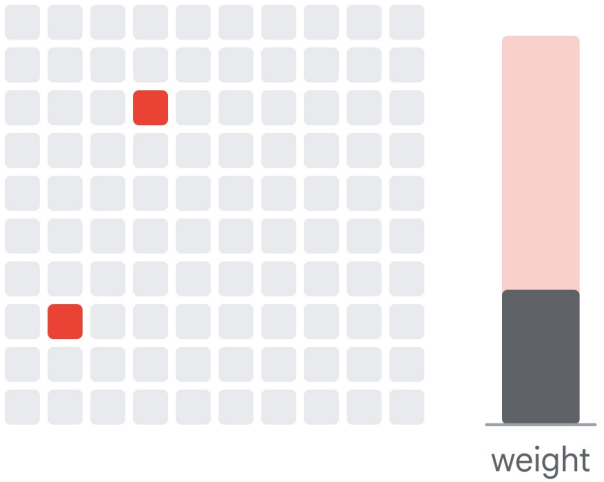
N = 200, df = 52

✓ **Same match, same weight**

Melnik, Raghavan, Yang & Garcia-Molina (2001)

Shard 2

100 documents



weight

df = 2 of 100

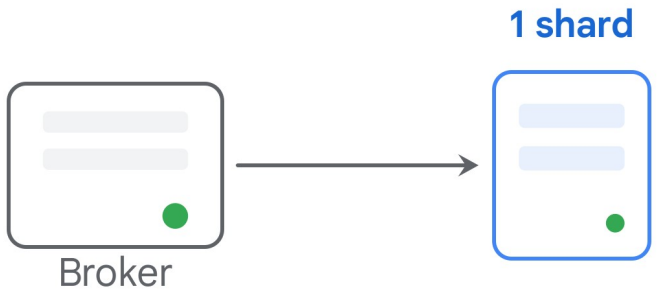
idf = log(100 / 2) = **1.70**

idf = log(200 / 52) = **0.59** ✓

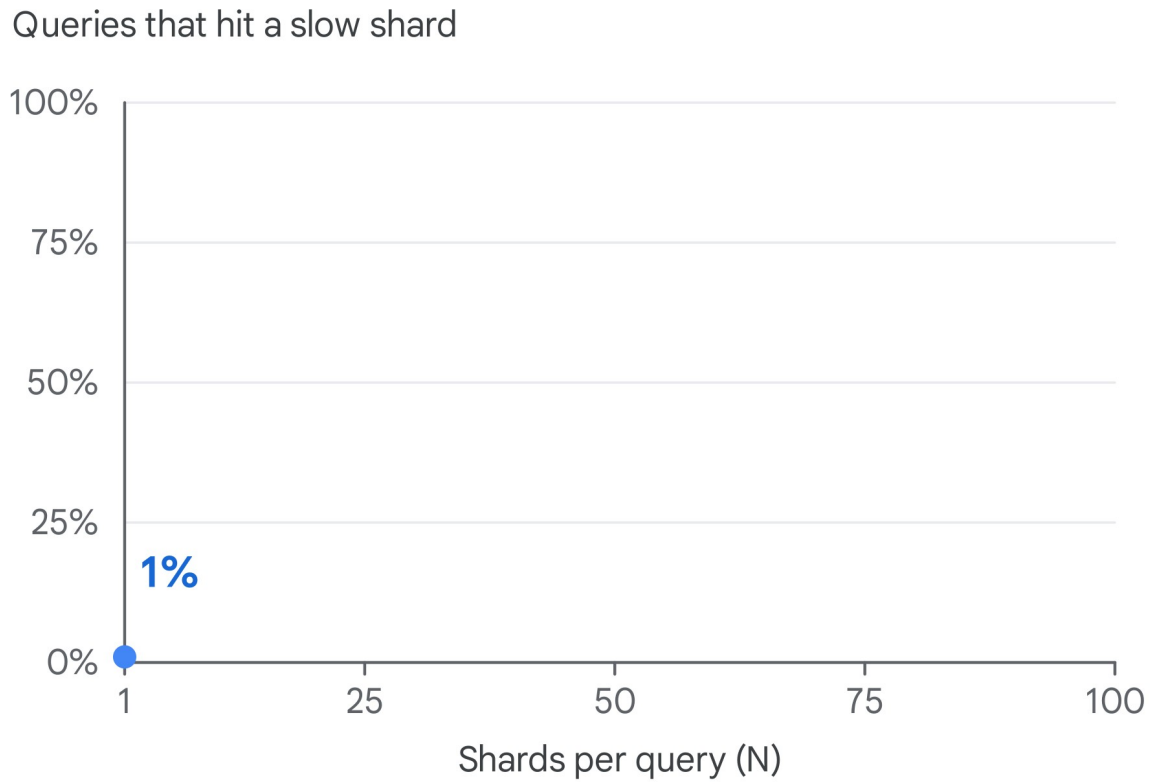
The price of fan-out: tail latency

Èric

Problem | **Principles** | Example | Trends | More

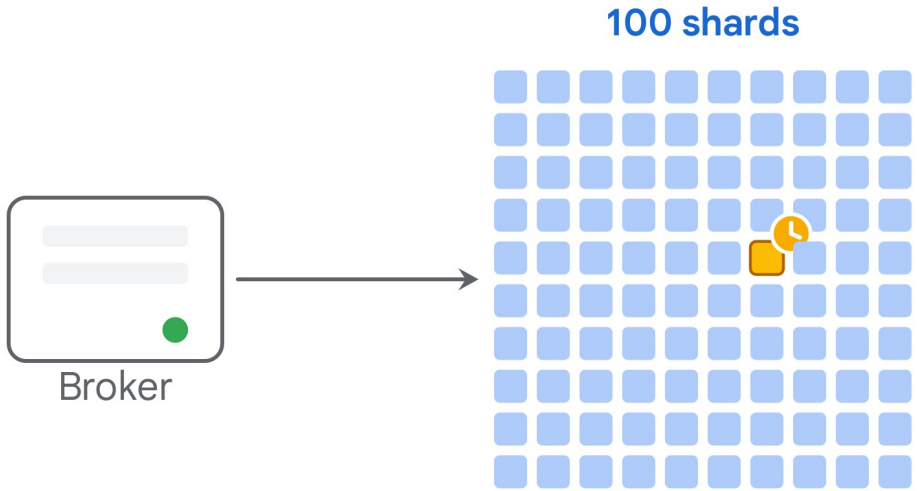


Each shard is slow 1% of the time



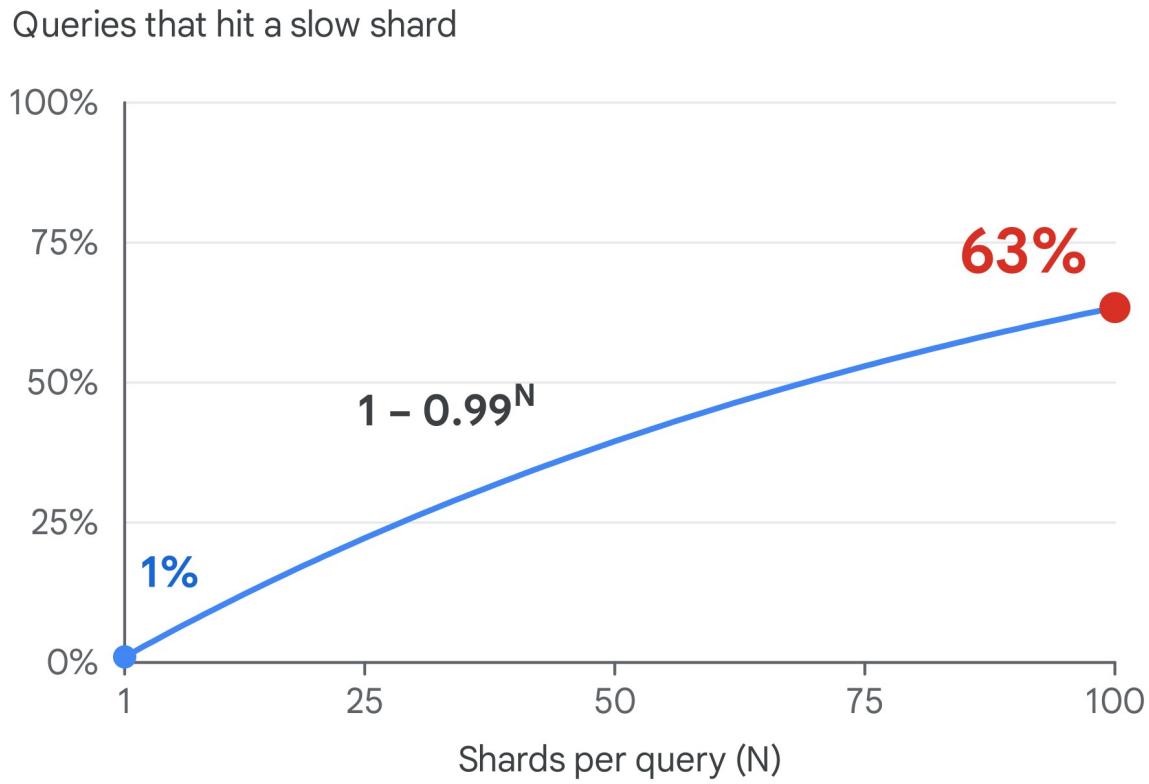
Dean & Barroso (2013), The tail at scale

The price of fan-out: tail latency



Each shard is slow 1% of the time

The broker waits for the slowest one



Dean & Barroso (2013), The tail at scale

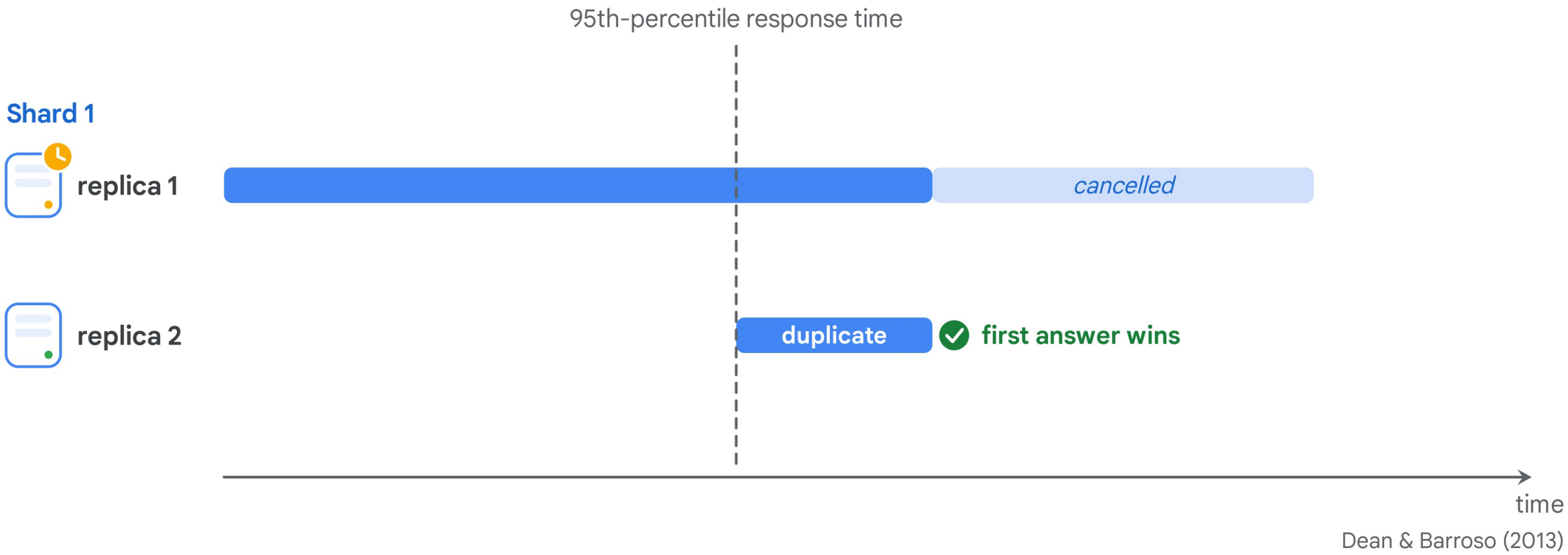
Most queries get the bad luck of some shard: the tail at scale.

Fixing the tail: a hedged request

Èric

Problem | **Principles** | Example | Trends | More

Duplicate after the 95th-percentile time, take the first answer



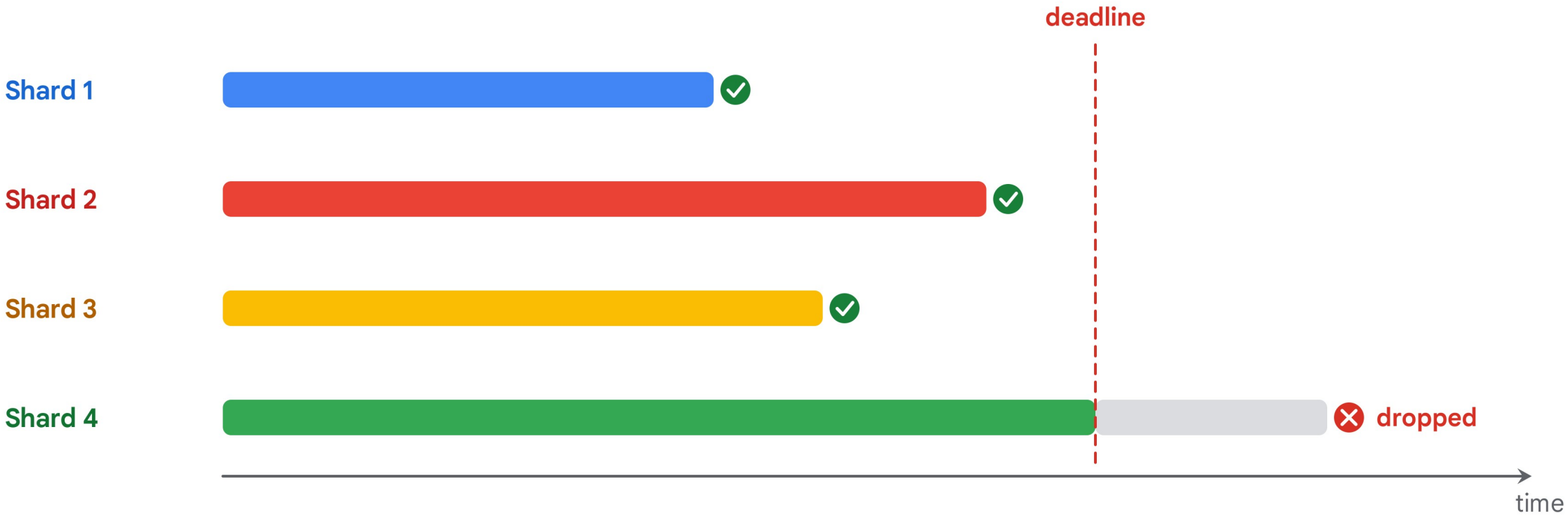
Dean & Barroso (2013)

Fixing the tail: stop at a deadline

Èric

Problem | **Principles** | Example | Trends | More

Answer with the shards that made it: slightly less complete, much faster

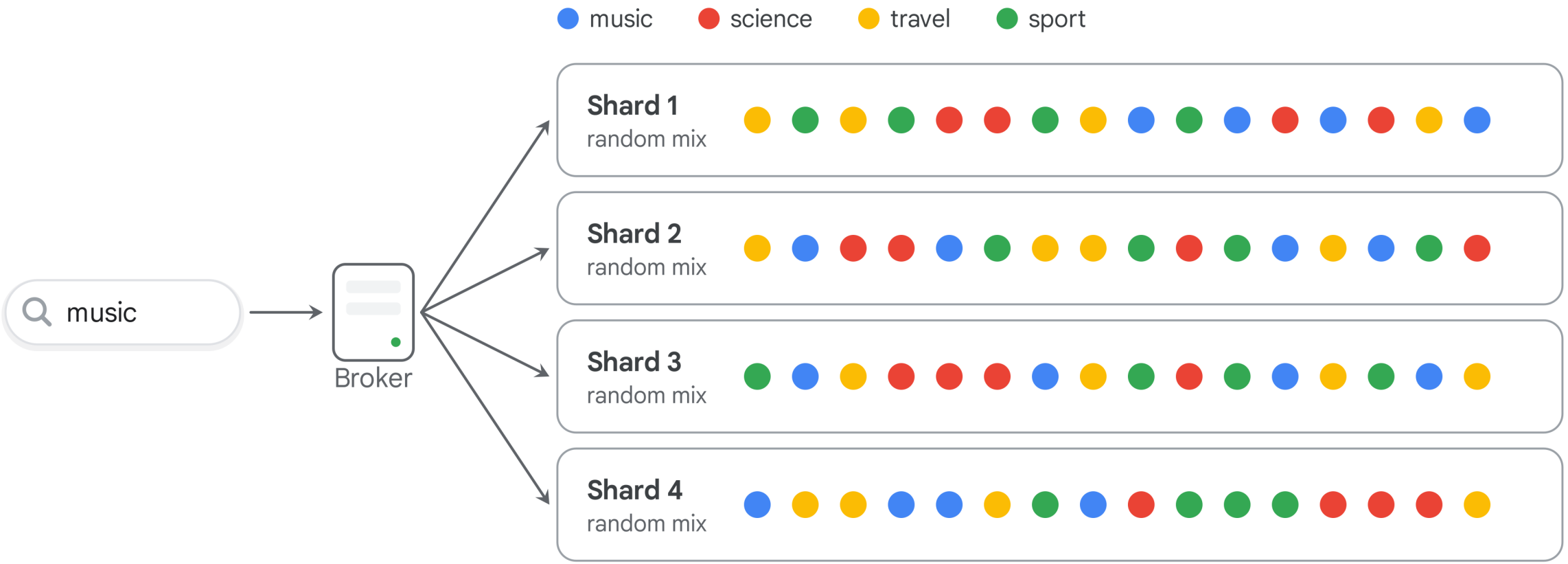


Dean & Barroso (2013)

Selective search: don’t ask everyone

Èric

Problem | **Principles** | Example | Trends | More



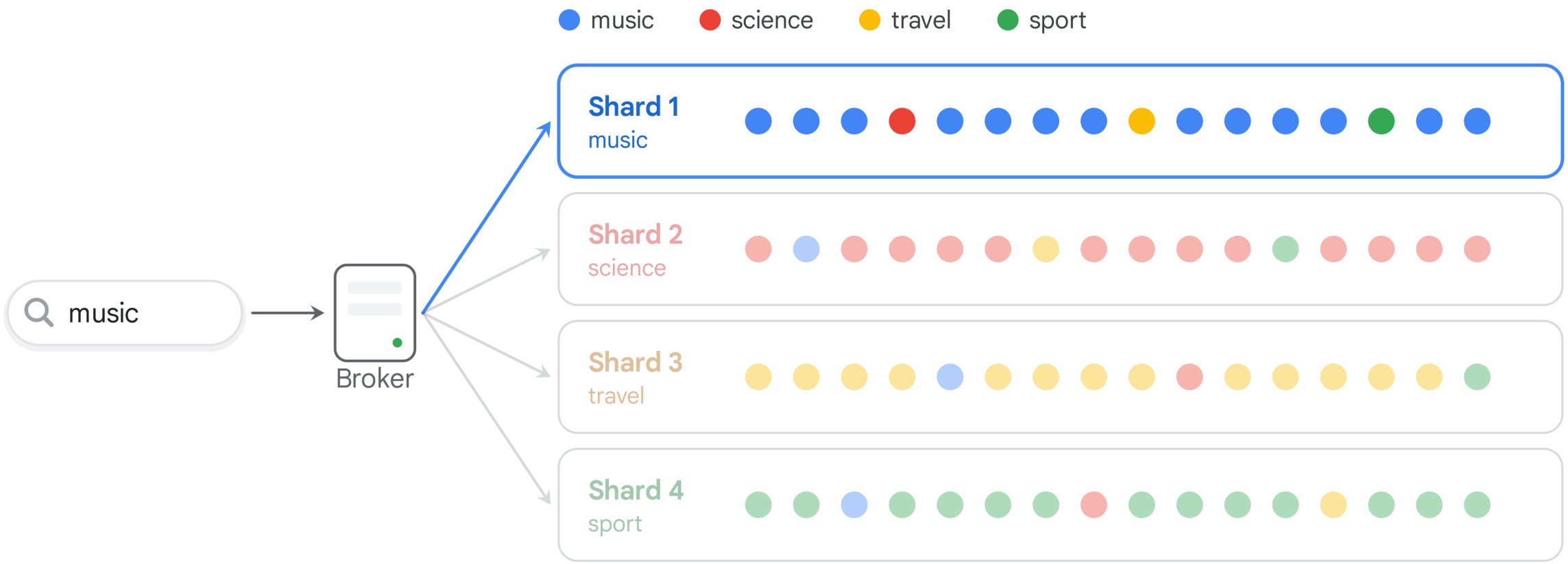
Random assignment: ask all 4 shards

Kulkarni & Callan (2015), Kim et al. (2017)

Selective search: don’t ask everyone

Èric

Problem | **Principles** | Example | Trends | More



Topical shards: ask 1 of 4, results close to asking all

Kulkarni & Callan (2015), Kim et al. (2017)

One query, step by step: a toy cluster

lamine yamal

Cache
lamine yamal

Broker

machine

document

waiting requests



toy example, invented scores

Step 0: normalise, then check the cache

Oriol O

Problem | Principles | **Example** | Trends | More

lamine yamal

Cache
lamine yamal

Broker

0 Cache first
lowercase, split into tokens
a hit skips the cluster

Shard 1
sports news
A B C
N1 Yamal: best young player
N2 Barça squad list
N3 Weekend fixtures

Shard 2
encyclopedic
D E F
E1 Lamine Yamal: biography
E2 FC Barcelona squad
E3 Camp Nou stadium

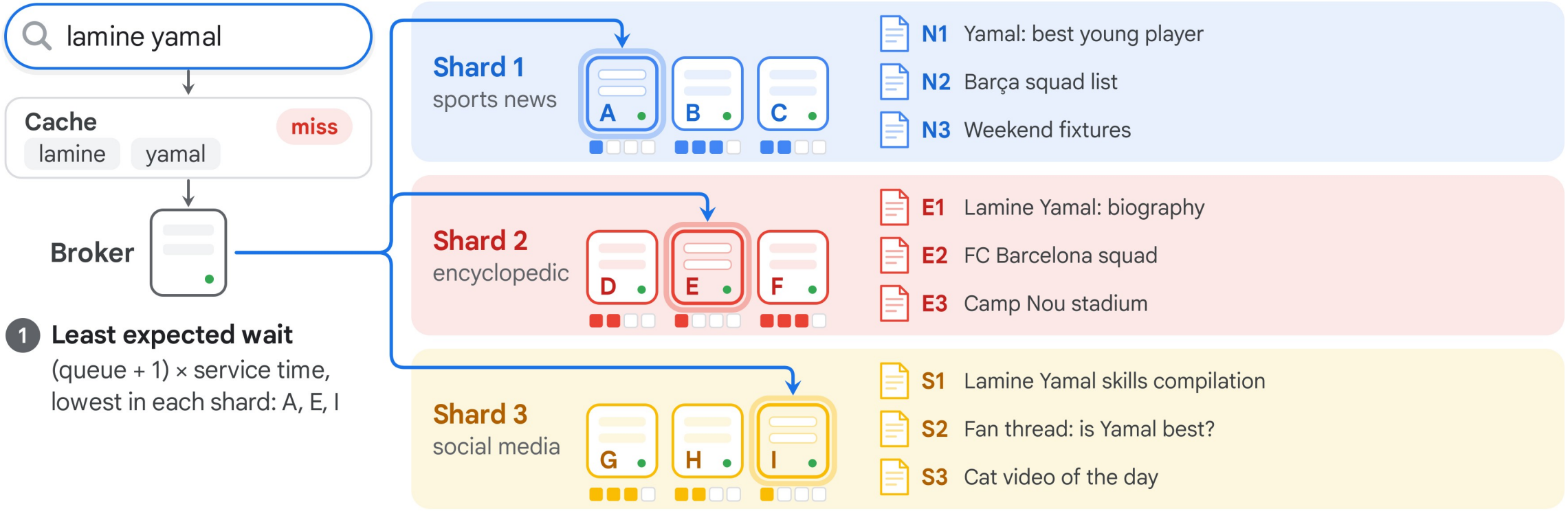
Shard 3
social media
G H I
S1 Lamine Yamal skills compilation
S2 Fan thread: is Yamal best?
S3 Cat video of the day

toy example, invented scores

Step 1: route to the least busy replicas

Oriol O

Problem | Principles | **Example** | Trends | More



toy example, invented scores

Step 2: each shard searches locally

Oriol

Problem | Principles | **Example** | Trends | More

lamine yamal

Cache — miss
lamine | yamal

Broker

2 Local search
look up both terms, intersect, score, send back the top 2

Shard 1
sports news

A B C

N1 Yamal: best young player
N2 Barça squad list
N3 Weekend fixtures

Top 2 from A
N1 8.1
N2 7.4

Shard 2
encyclopedic

D E F

E1 Lamine Yamal: biography
E2 FC Barcelona squad
E3 Camp Nou stadium

Waiting for E...

Shard 3
social media

G H I

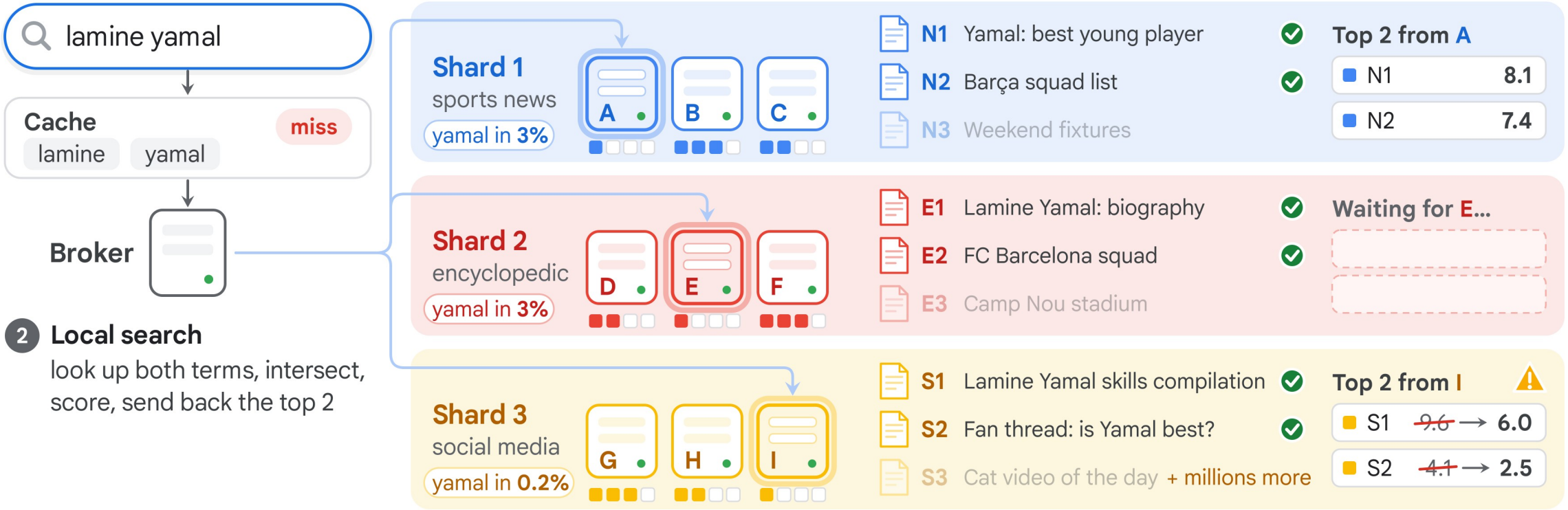
S1 Lamine Yamal skills compilation
S2 Fan thread: is Yamal best?
S3 Cat video of the day

Top 2 from I
S1 9.6
S2 4.1

toy example, invented scores

Step 2: the merging trap, in our example

Oriol O



toy example, invented scores

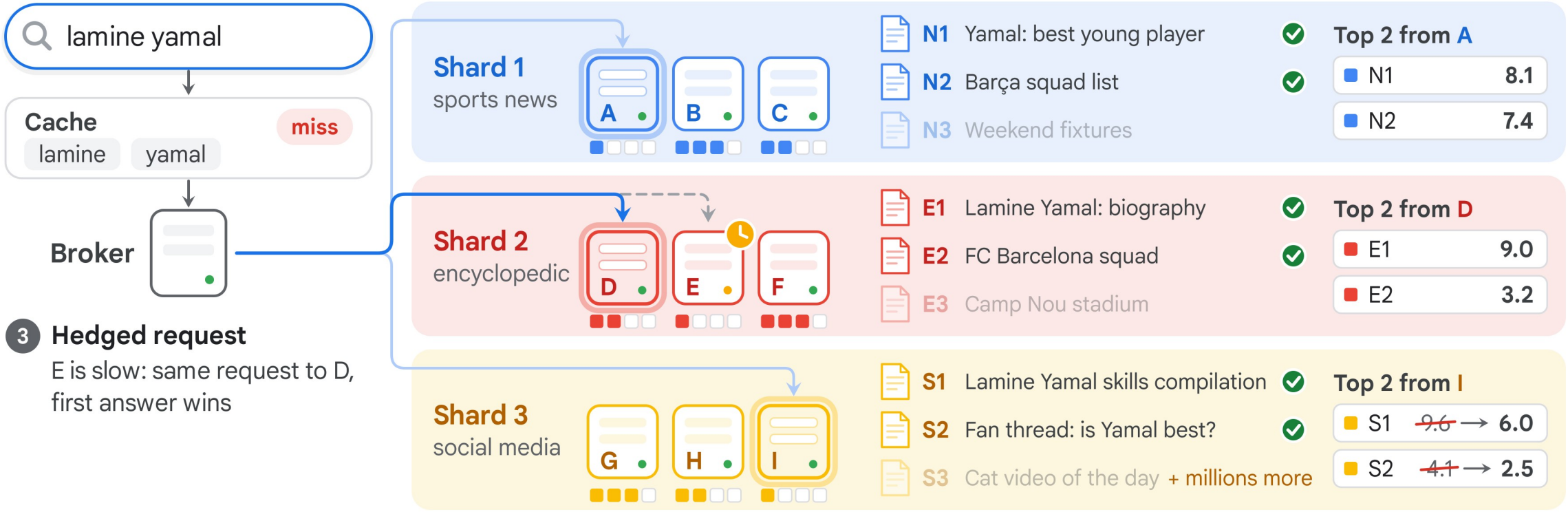
The merging trap

'yamal' is rare among shard 3's posts, so its local IDF inflates the scores. Global statistics fix it, and shards 1–2 change only slightly.

Step 3: a slow machine, a hedged request

Oriol O

Problem | Principles | **Example** | Trends | More

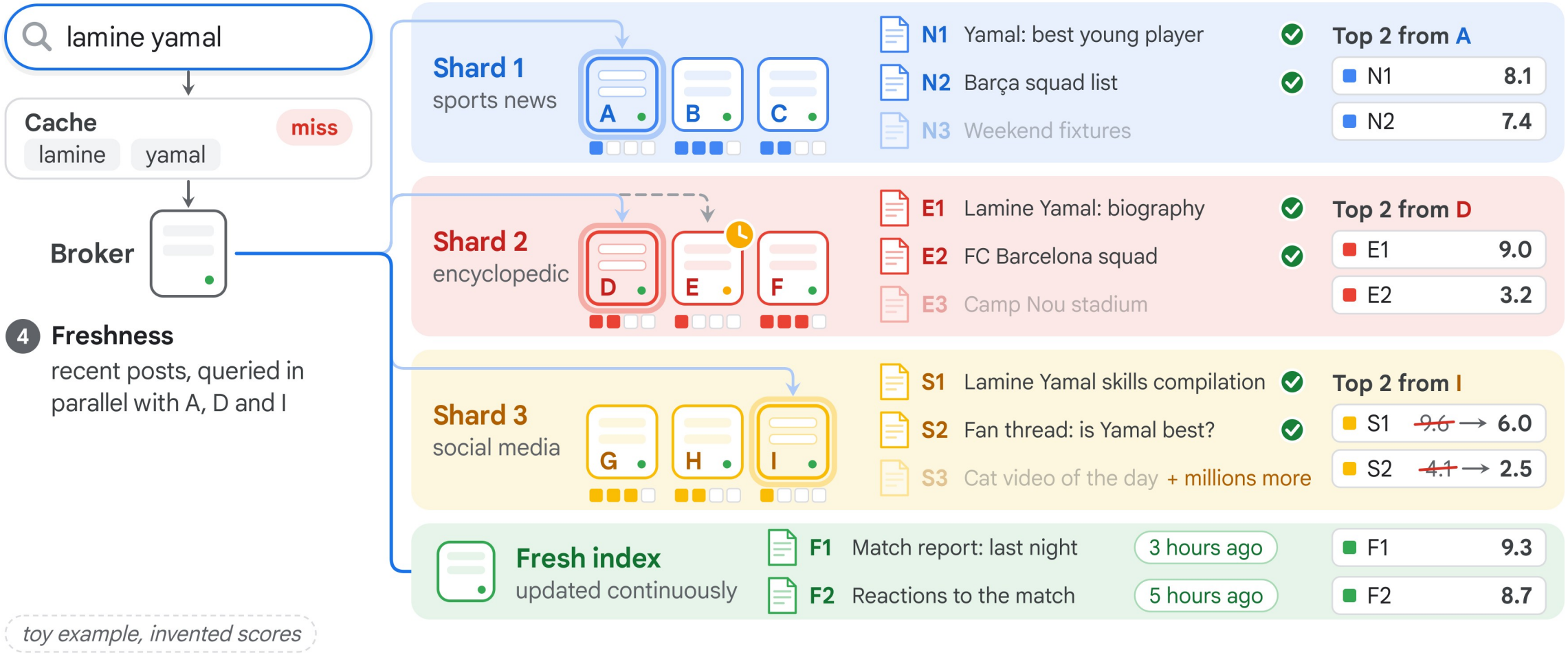


toy example, invented scores

Step 4: a fresh index answers too

Oriol

Problem | Principles | **Example** | Trends | More

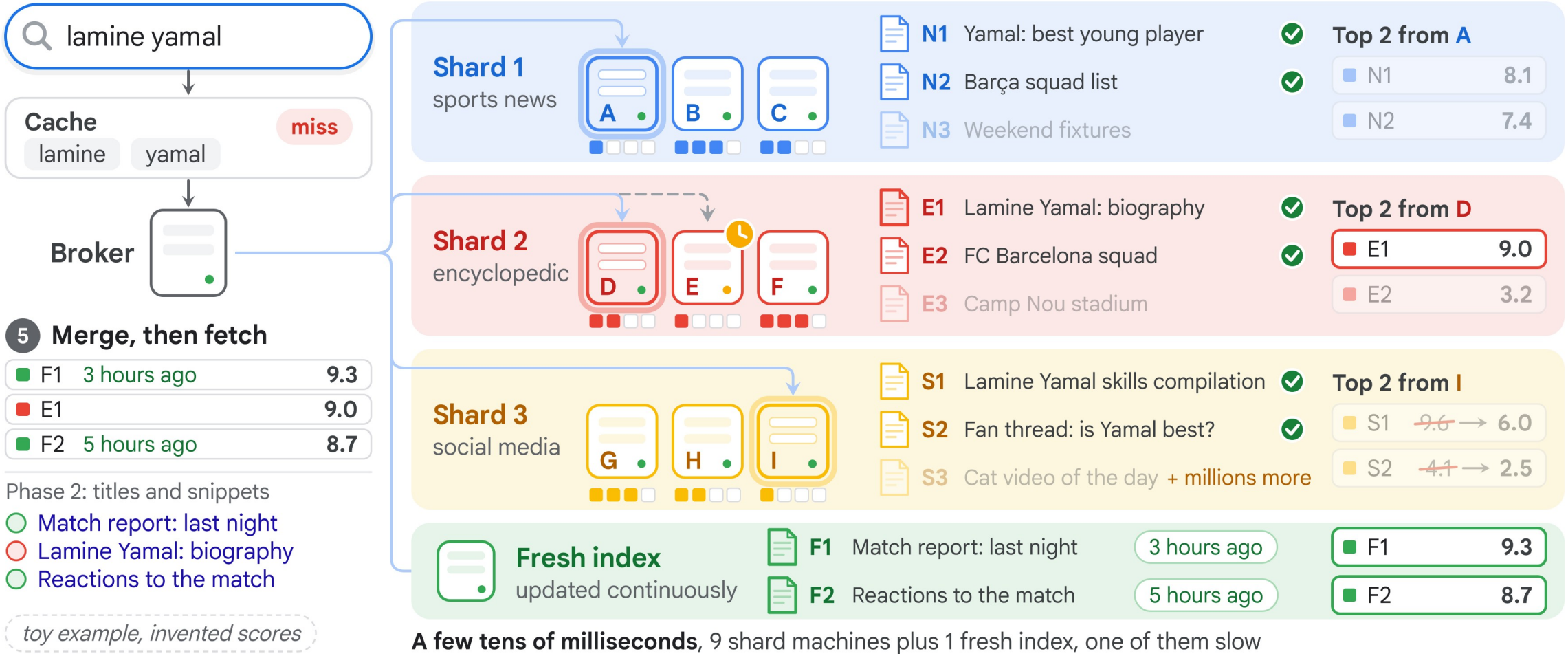


toy example, invented scores

Step 5: merge, then fetch the snippets

Oriol O

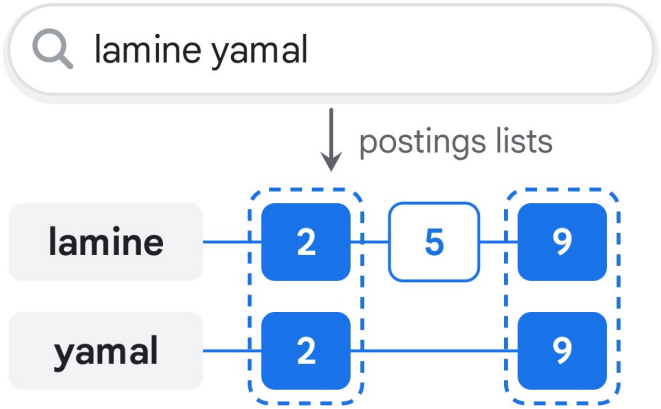
Problem | Principles | **Example** | Trends | More



A few tens of milliseconds, 9 shard machines plus 1 fresh index, one of them slow

Trend 1: what is matched is changing

1 Words



pages containing both words

2 Language and intent

3 Meaning as vectors

over the last decade

Trend 1: what is matched is changing

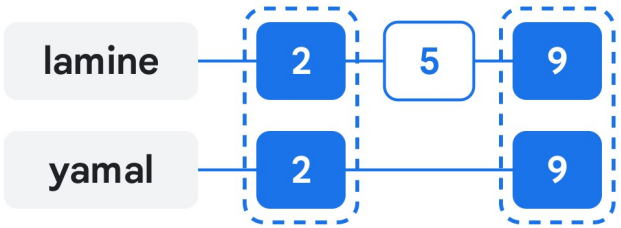
Oriol O

Problem | Principles | Example | **Trends** | More

1 Words

lamine yamal

postings lists



pages containing both words

2 Language and intent

lamine yamal

learned models

person: footballer

intent: latest news

plus many ranking signals

3 Meaning as vectors

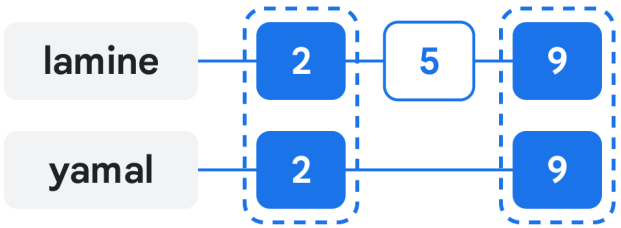
over the last decade

Trend 1: what is matched is changing

1 Words

lamine yamal

postings lists



pages containing both words

2 Language and intent

lamine yamal

learned models

person: footballer

intent: latest news

plus many ranking signals

3 Meaning as vectors

lamine yamal

[0.12, −0.83, ...]

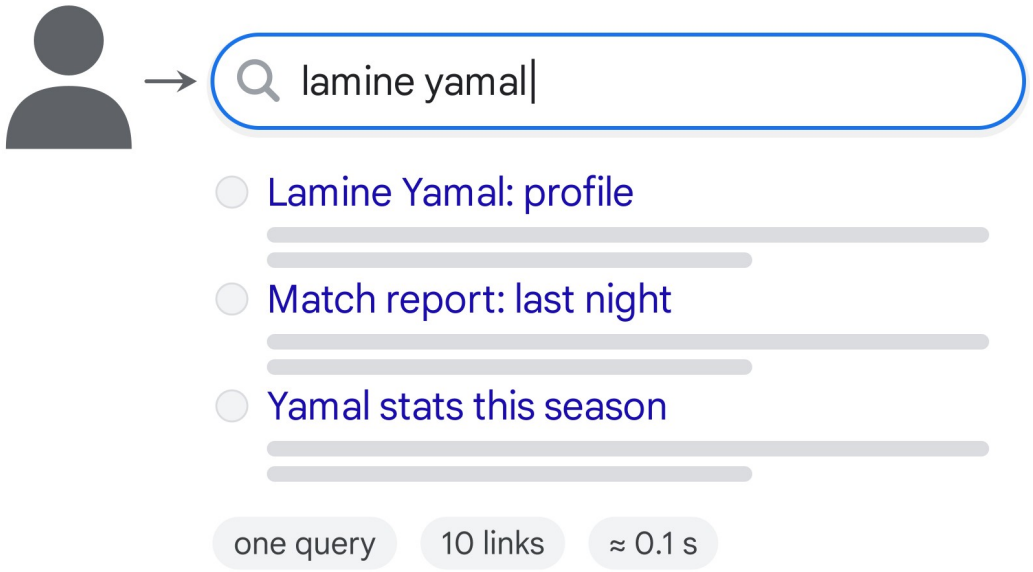


with or instead of postings lists

over the last decade

Trend 2: who is searching is changing

A person

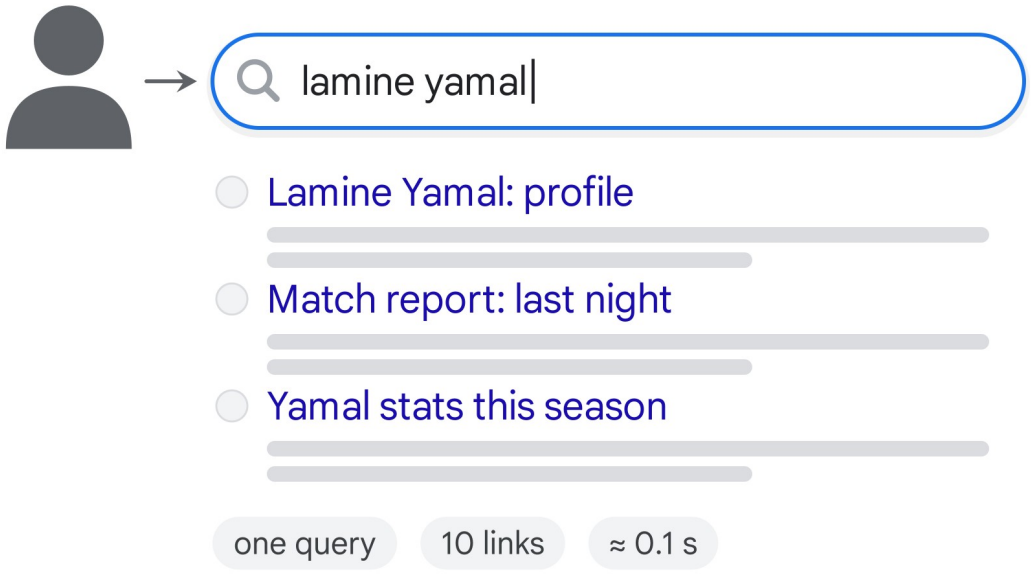


Trend 2: who is searching is changing

Oriol O

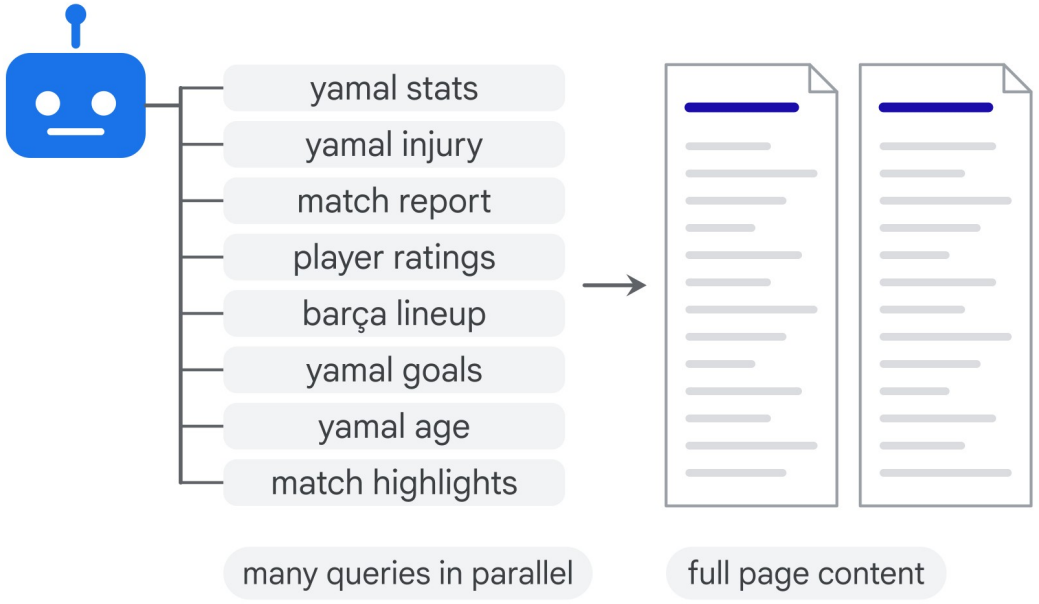
Problem | Principles | Example | **Trends** | More

A person



An agent

e.g. Exa, built for agents



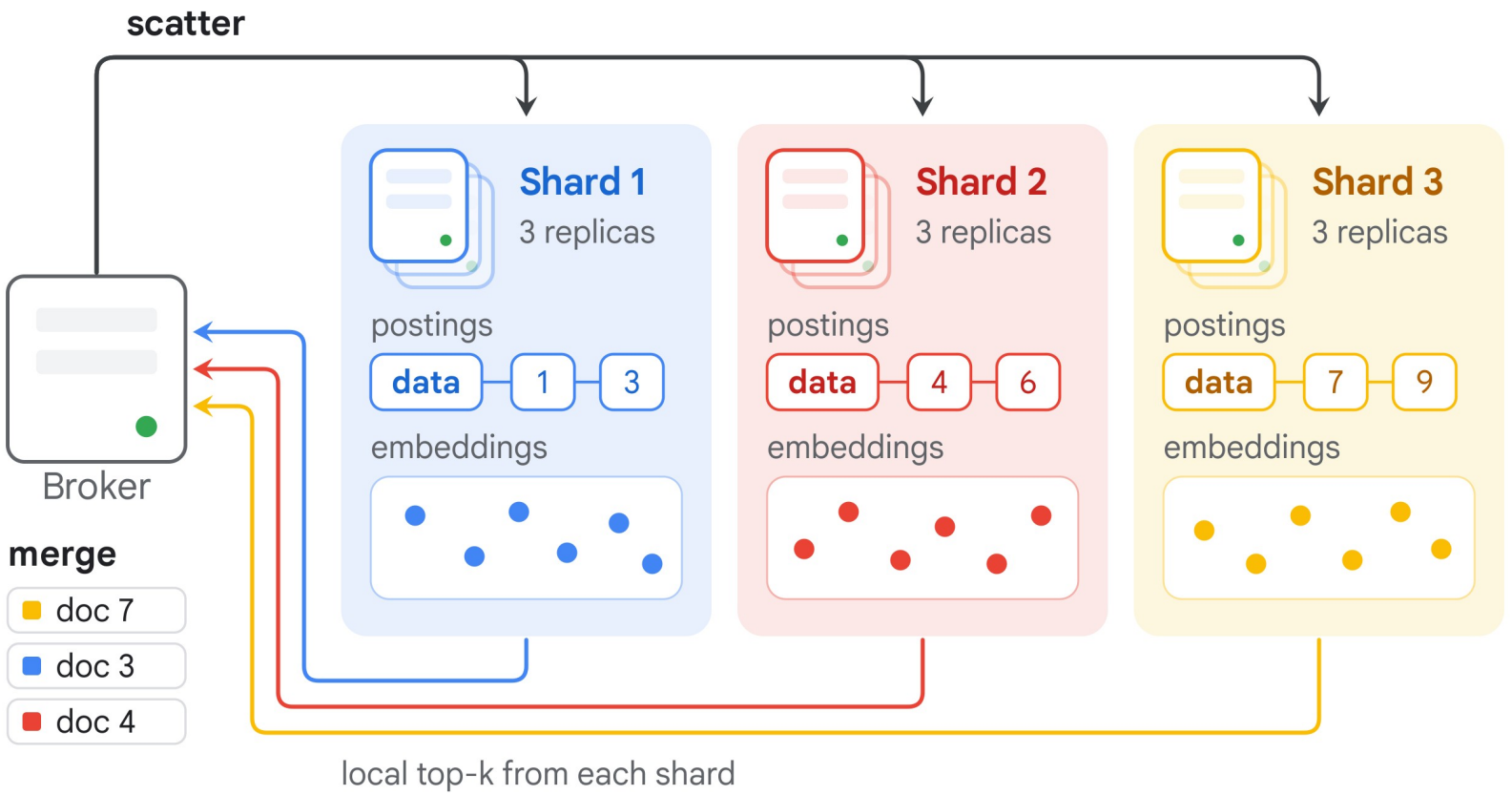
For the cluster ■■■■□ more queries per second → more replicas

heavier phase two: full pages, not snippets

What stays the same

Oriol O

Problem | Principles | Example | **Trends** | More



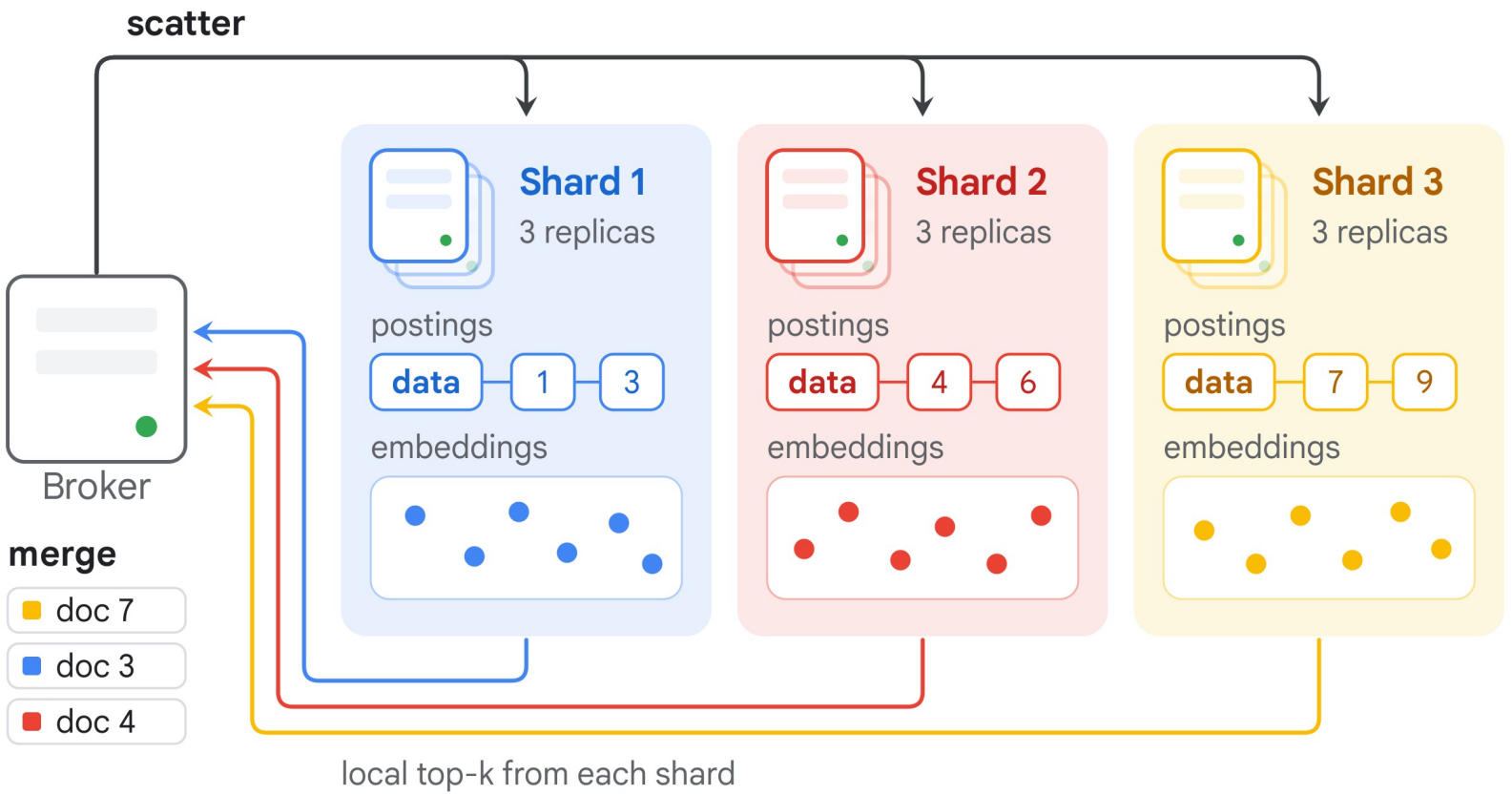
- postings lists or embeddings: **the same pattern**

arXiv 2507.06653, Elastic Search Labs

What stays the same, and what changes

Oriol O

Problem | Principles | Example | **Trends** | More



✅ postings lists or embeddings: **the same pattern**

What changes

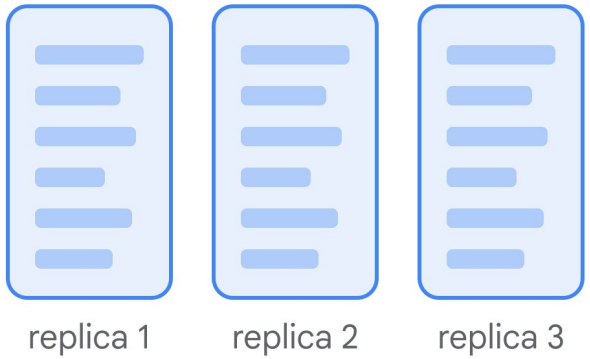
⚠️ **Which shard gets each vector?**
at random (ask all), or by cluster (ask a few, like selective search)

⚠️ **Fusing keyword and vector scores**
normalise, or fuse by rank

arXiv 2507.06653, Elastic Search Labs

Limits: every tool has a price

1 Replicas cost memory



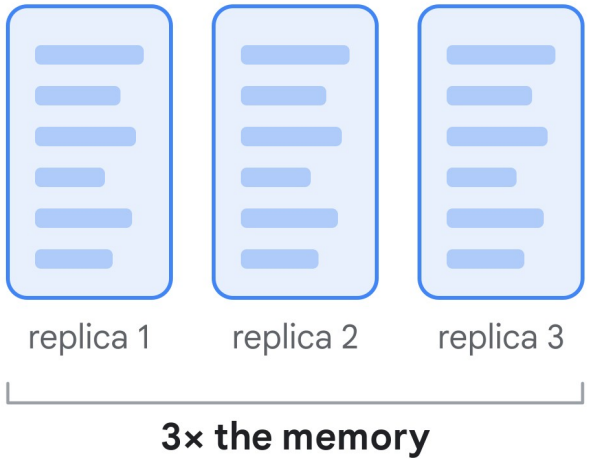
every copy is a full copy

2 More shards, worse tail

3 Selective search can miss

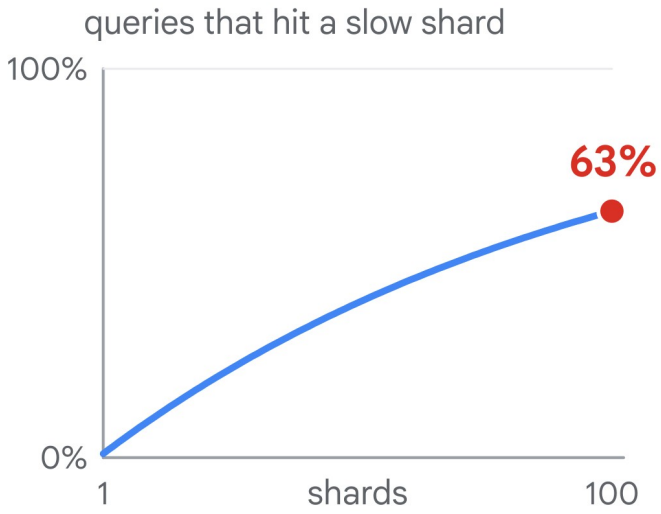
Limits: every tool has a price

1 Replicas cost memory



every copy is a full copy

2 More shards, worse tail

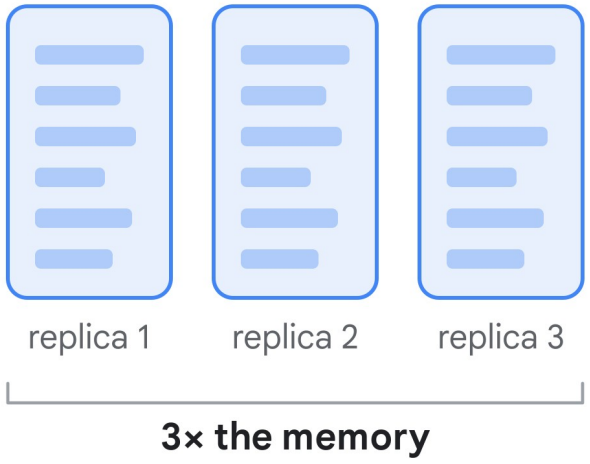


the slowest shard sets the pace

3 Selective search can miss

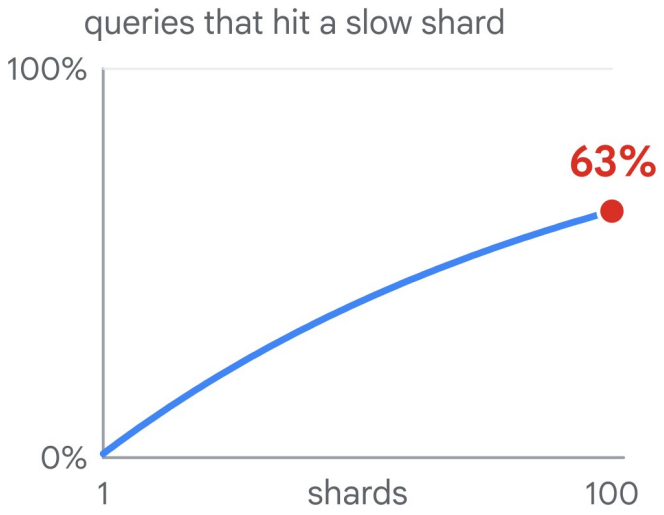
Limits: every tool has a price

1 Replicas cost memory



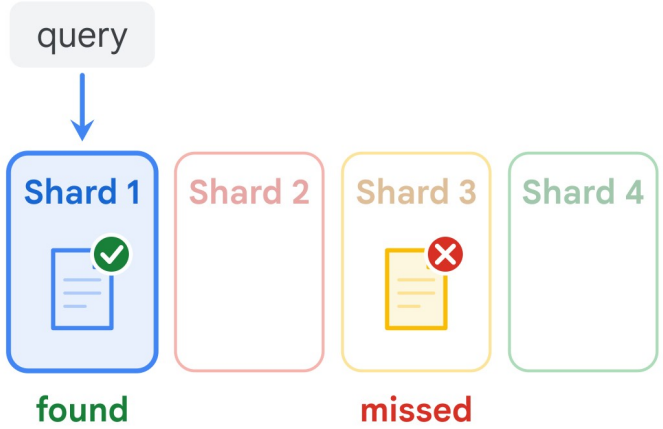
every copy is a full copy

2 More shards, worse tail



the slowest shard sets the pace

3 Selective search can miss



relevant pages in shards not asked

Sources

Oriol O

Problem Principles Example Trends **More**

12 results

IRWA course slides, UPF, 2026

Indexing and searching (theory session 5)

Inverted index, document partitioning, the 2016 Gartner server estimate.

Brin & Page, Computer Networks and ISDN Systems 30, 1998, doi:10.1016/S0169-7552(98)00110-X

The anatomy of a large-scale hypertextual Web search engine

The 1998 prototype (24 million pages).

Barroso, Dean & Hölzle, IEEE Micro 23(2), 2003, doi:10.1109/MM.2003.1196112

Web search for a planet: the Google cluster architecture

2003 scale, random document shards, replica pools, two phases.

Moffat, Webber, Zobel & Baeza-Yates, Information Retrieval 10(3), 2007, ...

A pipelined architecture for distributed text query evaluation

Term vs document partitioning.

Suresh, Canini, Schmid & Feldmann, USENIX NSDI, 2015

C3: cutting tail latency in cloud data stores via adaptive replica selection

Choosing replicas by expected wait (data stores).

Melnik, Raghavan, Yang & Garcia-Molina, ACM TOIS 19(3), 2001

Building a distributed full-text index for the Web

Global statistics across shards.

Dean & Barroso, Communications of the ACM 56(2), 2013, doi:10.1145/2408776.2408794

The tail at scale

Tail latency, hedged requests, partial answers.

Kulkarni & Callan, ACM TOIS 33(4), 2015

Selective search: efficient and effective search of large textual collections

Topical shards.

Kim, Callan, Culpepper & Moffat, Information Retrieval Journal 20(3), 2017

Efficient distributed selective search

Selective search on a cluster.

arXiv:2507.06653, 2025

Towards efficient and scalable distributed vector search with RDMA

Vector search: shards, scatter-gather, merge.

Elastic Search Labs blog, elastic.co/search-labs

Hybrid search with the linear retriever, and Hybrid search in Elasticsearch

Fusing keyword and vector scores.

exa.ai/docs, accessed 20 October 2026

Exa documentation

A web search tool designed for AI agents (Exa's own description).

How we used LLMs

Oriol O

Problem Principles Example Trends More

AI Overview

LLMs were assistants for reading, shortening and drawing: the thinking and the decisions were ours.

- **Model:** mostly Claude Opus 5.5 and Sonnet 5.5 (Anthropic), through Claude Code, a general-purpose agent. We also tried other assistants, but Claude Code was the main one.
- **Used for:** asking about topics in the papers, shortening the spoken script, and writing the code that draws the figures.
- **Proudly done by us:** the structure and the contents, finding, reading and understanding the papers, the Google-like theme and the ideas behind its design, and iterating on every diagram to make it accurate, not only good-looking.

AI responses may include mistakes.